

***Ahmedabad Institute of Technology***

***CE & IT Department***

# ***Hand Book***

**Compiler Design (2170701)**

**Year: 2020**

**Prepared By: Prof. Neha Prajapati**

## 1. Explain Translation Process or Translator.

- A translator is a kind of program that takes one form of program as input and converts it into another form.
- The input is called *source program* and output is called *target program*.
- The source language can be assembly language or higher level language like C, C++, FORTRAN, etc...
- There are three types of translators,
  1. Compiler
  2. Interpreter
  3. Assembler

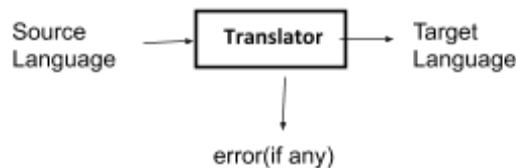


Fig.1.1. A Translator

## 2. What is compiler?

A compiler is a program that reads a program written in one language and translates into an equivalent program in another language.



Fig.1.2. A Compiler

Major functions done by compiler:

- Compiler is used to convert one form of program to another.
- A compiler should convert the source program to a target machine code in such a way that the generated target code should be easy to understand.
- Compiler should preserve the meaning of source code.
- Compiler should report errors that occur during compilation process.

- The compilation must be done efficiently.

### 3. Explain phases of the compiler

There are mainly two parts of compilation process.

1. **Analysis phase:** The main objective of the analysis phase is to break the source code into parts and then arranges these pieces into a meaningful structure.

Analysis part is divided into three sub parts,

- Lexical analysis
- Syntax analysis
- Semantic analysis

#### Lexical analysis:

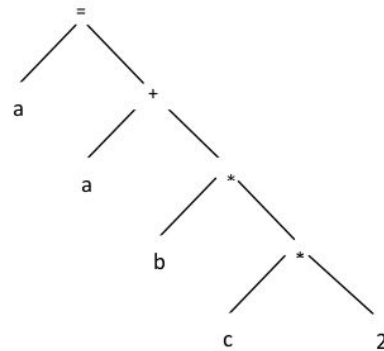
- Lexical analysis is also called linear analysis or scanning.
- Lexical analyzer reads the source program and then it is broken into stream of units. Such units are called token.
- Then it classifies the units into different lexical classes. E.g. id's, constants, keyword etc...and enters them into different tables.
- For example, in lexical analysis the assignment statement **a = a + b \* c \* 2** would be grouped into the following tokens:

|   |                     |
|---|---------------------|
| a | Identifier 1        |
| = | Assignment sign     |
| a | Identifier 1        |
| + | The plus sign       |
| b | Identifier 2        |
| * | Multiplication sign |
| c | Identifier 3        |
| * | Multiplication sign |

|   |          |
|---|----------|
| 2 | Number 2 |
|---|----------|

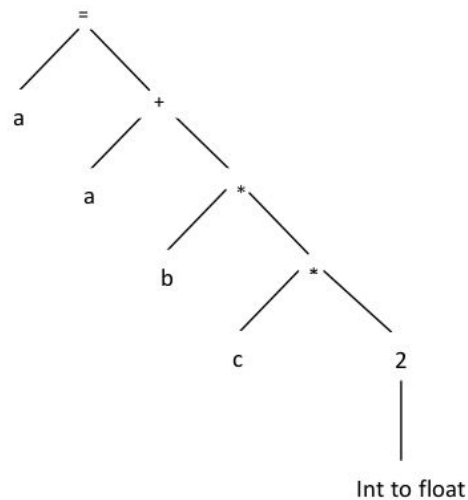
#### Syntax Analysis:

- Syntax analysis is also called hierarchical analysis or parsing.
- The syntax analyzer checks each line of the code and spots every tiny mistake that the programmer has committed while typing the code.
- If code is error free then syntax analyzer generates the tree.



#### Semantic analysis:

- Semantic analyzer determines the meaning of a source string.
- For example matching of parenthesis in the expression, or matching of if..else statement or performing arithmetic operation that are type compatible, or checking the scope of operation.



2. **Synthesis phase:** synthesis part is divided into three sub parts,
  - I. Intermediate code generation

- II. Code optimization
- III. Code generation

**Intermediate code generation:**

- The intermediate representation should have two important properties, it should be

easy to produce and easy to translate into target program.

- We consider intermediate form called “three address code”.
- Three address code consist of a sequence of instruction, each of which has at most three operands.
- The source program might appear in three address code as,

```
t1= int to real(2)
t2= id3 * t1
t3= t2 * id2
t4= t3 + id1
id1= t4
```

#### **Code optimization:**

- The code optimization phase attempt to improve the intermediate code.
- This is necessary to have a faster executing code or less consumption of memory.
- Thus by optimizing the code the overall running time of a target program can be improved.

```
t1= id3 * 2.0
t2= id2 * t1
id1 = id1 + t2
```

#### **Code generation:**

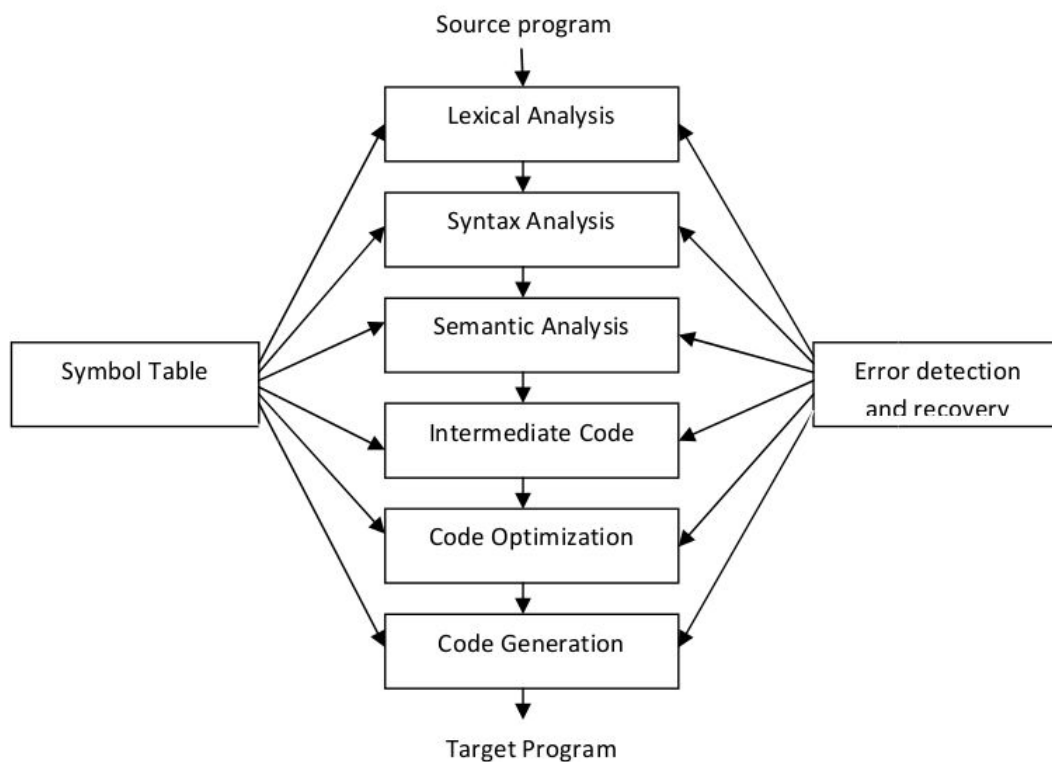
- In code generation phase the target code gets generated. intermediate code

The instructions are translated into sequence of machine instruction.

```
MOV id3, R1
MUL #2.0, R1
MOV id2, R2
MUL R2, R1
MOV id1, R2
ADD R2, R1
MOV R1, id1
```

## Symbol Table

- A **symbol table** is a data structure used by a language translator such as a compiler or interpreter.
- It is used to store names encountered in the source program, along with the relevant attributes for those names.
- Information about following entities is stored in the symbol table.
  - Variable/Identifier
  - Procedure/function
  - Keyword
  - Constant
  - Class name
  - Label name



## 4. The context of a Compiler OR Cousins of compiler.

In addition to a compiler, several other programs may be required to create an executable target program.

## **Preprocessor**

Preprocessor produces input to compiler. They may perform the following functions,

- Macro processing: A preprocessor may allow user to define macros that are shorthand for longer constructs.
- File inclusion: A preprocessor may include the header file into the program text.
- Rational preprocessor: Such a preprocessor provides the user with built in macro for construct like while statement or if statement.
- Language extensions: this processors attempt to add capabilities to the language by what amount to built-in macros. Ex: the language equal is a database query language embedded in C. statement beginning with ## are taken by preprocessor to be database access statement unrelated to C and translated into procedure call on routines that perform the database access.

## **Assembler**

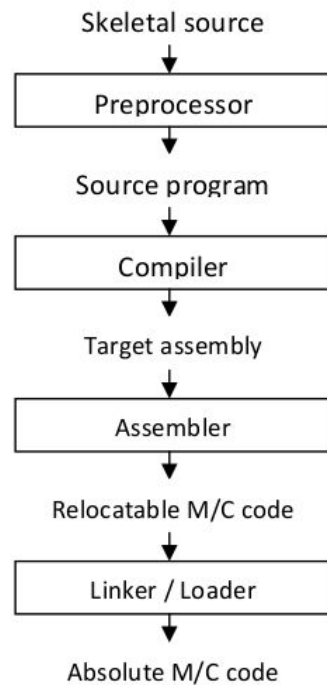
Assembler is a translator which takes the assembly program as an input and generates the machine code as a output. An assembly is a mnemonic version of machine code, in which names are used instead of binary codes for operations.

## **Linker**

Linker allows us to make a single program from a several files of relocatable machine code. These file may have been the result of several different compilation, and one or more may be library files of routine provided by a system.

## **Loader**

The process of loading consists of taking relocatable machine code, altering the relocatable address and placing the altered instructions and data in memory at the proper location.



**5. Explain front end and back end in brief. (Grouping of phases)**

The phases are collected into a front end and back end.

**Front end**

- The front end consist of those phases, that depends primarily on source language and largely independent of the target machine.
- Front end includes lexical analysis, syntax analysis, semantic analysis, intermediate code generation and creation of symbol table.
- Certain amount of code optimization can be done by front end.

**Back end**

- The back end consists of those phases, that depends on target machine and do not depend on source program.
- Back end includes code optimization and code generation phase with necessary error handling and symbol table operation.

**6. What is the pass of compiler? Explain how the single and multi- pass compilers work? What is the effect of reducing the number of passes?**

One complete scan of a source program is called pass.

Pass include reading an input file and writing to the output file.

In a single pass compiler analysis of source statement is immediately followed by synthesis of equivalent target statement.

It is difficult to compile the source program into single pass due to:

**Forward reference:** a forward reference of a program entity is a reference to the entity which precedes its definition in the program.

This problem can be solved by postponing the generation of target code until more information concerning the entity becomes available.

It leads to multi pass model of compilation.

In Pass I: Perform analysis of the source program and note relevant information.

In Pass II: Generate target code using information noted in pass I.

#### **Effect of reducing the number of passes**

It is desirable to have a few passes, because it takes time to read and write intermediate file.

On the other hand if we group several phases into one pass we may be forced to keep the entire program in the memory. Therefore memory requirement may be large.

## **7. Write difference of following:**

- Compiler and Interpreter
- Compiler and Assembler
- Single Pass Compiler and Multipass compiler
- Phase and Pass

#### ***Compiler v/s Interpreter***

| No. | Compiler                                   | Interpreter                                       |
|-----|--------------------------------------------|---------------------------------------------------|
| 1   | Compiler takes entire program as an input. | Interpreter takes single instruction as an input. |
| 2   | Intermediate code is generated.            | No Intermediate code is generated.                |
| 3   | Memory requirement is more.                | Memory requirement is less.                       |

|   |                                                     |                                                       |
|---|-----------------------------------------------------|-------------------------------------------------------|
| 4 | Error is displayed after entire program is checked. | Error is displayed for every instruction interpreted. |
| 5 | Example: C compiler                                 | Example: BASIC                                        |

**Table 1.1 Difference between Compiler & Interpreter**

### ***Compiler v/s Assembler***

| No. | Compiler                                                                                                                   | Assembler                                                                                                                   |
|-----|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1   | It translates higher level language to machine code.                                                                       | It translates mnemonic operation code to machine code.                                                                      |
| 2   | Types of compiler, <ul style="list-style-type: none"> <li>• Single pass compiler</li> <li>• Multi pass compiler</li> </ul> | Types of assembler, <ul style="list-style-type: none"> <li>• Single pass assembler</li> <li>• Two pass assembler</li> </ul> |
| 3   | Example: C compiler                                                                                                        | Example: 8085, 8086 instruction set                                                                                         |

**Table 1.2 Difference between Compiler & Assembler**

### ***Single pass compiler v/s Multi pass Compiler***

| No. | Single pass compiler                                                                                      | Multi pass compiler                                                                                                            |
|-----|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1   | A one-pass compiler is a compiler that passes through the source code of each compilation unit only once. | A multi-pass compiler is a type of compiler that processes the source code or abstract syntax tree of a program several times. |
| 2   | A one-pass compiler is faster than multi-pass compiler.                                                   | A multi-pass compiler is slower than single-pass compiler.                                                                     |
| 3   | One-pass compiler are sometimes called narrow compiler.                                                   | Multi-pass compilers are sometimes called wide compiler.                                                                       |
| 4   | Language like Pascal can be implemented with a single pass compiler.                                      | Languages like Java require a multi-pass compiler.                                                                             |

**Table 1.3 Difference between Single Pass Compiler & Multi Pass Compiler**

### ***Phase v/s Pass***

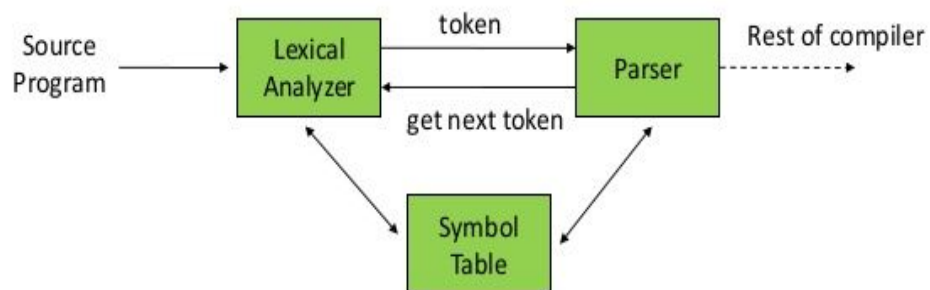
| No. | Phase | Pass |
|-----|-------|------|
|-----|-------|------|

|   |                                                                                                                                                          |                                                                                       |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1 | The process of compilation is carried out in various step is called phase.                                                                               | Various phases are logically grouped together to form a pass.                         |
| 2 | The phases of compilation are lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization and code generation. | The process of compilation can be carried out in a single pass or in multiple passes. |

**Table 1.3 Difference between Phase & Pass**

## Unit 2:lexical Analyzer

### 1. Role of lexical analysis and its issues.



- The lexical analyzer is the first phase of compiler. Its main task is to read the input characters and produce as output a sequence of tokens that the parser uses for syntax analysis.
- It is implemented by making lexical analyzer be a subroutine.
- Upon receiving a “get next token” command from parser, the lexical analyzer reads the input character until it can identify the next token.
- It may also perform secondary task at user interface.
- One such task is stripping out from the source program comments and white space in the form of blanks, tabs, and newline characters.
- Some lexical analyzer are divided into cascade of two phases, the first called scanning and second is “lexical analysis”.
- The scanner is responsible for doing simple task while lexical analysis does the more complex task.

#### **Issues in Lexical Analysis:**

There are several reasons for separating the analysis phase of compiling into lexical

analysis and parsing:

- Simpler design is perhaps the most important consideration. The separation of lexical analysis often allows us to simplify one or other of these phases.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

## 2. Tokens, Pattern and Lexeme

**Token:** Sequence of character having a collective meaning is known as *token*.

Typical tokens are,

1) Identifiers 2) keywords 3) operators 4) special symbols 5) constants

**Pattern:** The set of rules called *pattern* associated with a token.

**Lexeme:** The sequence of character in a source program matched with a pattern for a token is called lexeme.

| Token    | Lexeme              | Pattern                                |
|----------|---------------------|----------------------------------------|
| Const    | Const               | Const                                  |
| If       | If                  | If                                     |
| Relation | <, <=, =, >, >=, >  | < or <= or = or > or >= or >           |
| Id       | Pi, count, n, l     | letter followed by letters and digits. |
| Number   | 3.14159, 0, 6.02e23 | Any numeric constant                   |
| Literal  | "core"              | Any character between " and " except " |

### Example 1:

total = sum + 12.5

Tokens are: total (id),

= (relation) Sum (id)

+ (operator)

12.5 (num)

Lexemes are: total, =, sum, +, 12.5

**Example 2:**

```
if (price + gst – rebate <= 10.00) gift :=  
false
```

| Token      | lexeme | Informal description of pattern                                 |
|------------|--------|-----------------------------------------------------------------|
| if         | if     | if                                                              |
| Lparen     | (      | (                                                               |
| Identifier | price  | String consists of letters and numbers and starts with a letter |
| operator   | +      | +                                                               |
| identifier | gst    | String consists of letters and numbers and starts with a letter |
| operator   | -      | -                                                               |
| identifier | rebate | String consists of letters and numbers and starts with a letter |
| Operator   | <=     | Less than or equal to                                           |
| constant   | 10.00  | Any numeric constant                                            |
| rparen     | )      | )                                                               |
| identifier | gift   | String consists of letters and numbers and starts with a letter |
| Operator   | :=     | Assignment symbol                                               |
| identifier | false  | String consists of letters and numbers and starts with a letter |

**3. What is input buffering? Explain technique of buffer pair.**

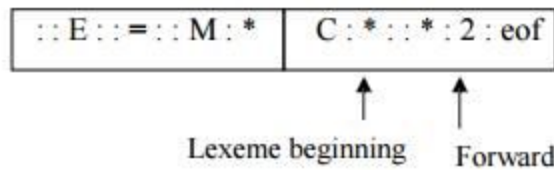
**OR Which technique is used for speeding up the lexical analyzer?**

There are mainly two techniques for input buffering,

- ✓ Buffer pair
- ✓ Sentinels

**1.Buffer pair:**

- The lexical analysis scans the input string from left to right one character at a time.
- So, specialized buffering techniques have been developed to reduce the amount of overhead required to process a single input character.
- We use a buffer divided into two N-character halves, as shown in figure 2.2. N is the number of character on one disk block.



- We read N input character into each half of the buffer.
- Two pointers to the input are maintained and string between current lexemes.
- Pointer *Lexeme Begin*, marks the beginning of the current lexeme.
- Pointer *Forward*, scans ahead until a pattern match is found.
- If forward pointer is at the end of first buffer half then second is filled with N input character.
- If forward pointer is at the end of second buffer half then first is filled with N input character.
- code to advance forward pointer is given below,

***if forward at end of first half then begin***

*reload  
second  
half;  
forward  
:=  
forward +  
1;*

***end***

***else if forward at end of second***

***half then begin***  
*reload first half;*

*move forward to beginning of first half;*

***end***

***else forward := forward + 1;***

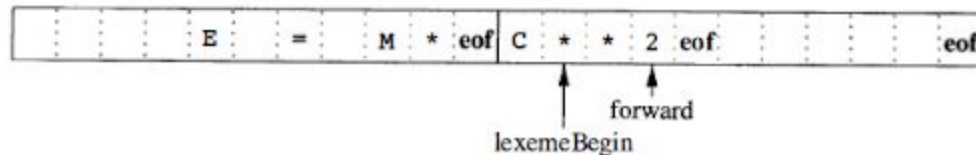
Once the next lexeme is determined, *forward* is set to character at its right end. Then, after the lexeme is recorded as an attribute value of a token returned to the parser, Lexeme Begin is set to the character immediately after the lexeme just found.

## 2. Sentinels:

- If we use the scheme of Buffer pairs we must check, each time we move the

forward pointer that we have not moved off one of the buffers; if we do, then we must reload the other buffer. Thus, for each character read, we make two tests.

- We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end. The sentinel is a special character that cannot be part of the source program, and a natural choice is the character **EOF**.



- Look ahead code with sentinels is given below:

```

forward := forward + 1;
if forward = eof then begin
    if forward at end of first
        half then begin
            reload second half;
            forward := forward + 1;
        end
    else if forward at the second
        half then begin reload
            first half;
            move forward to beginning of first half;
        end
    else terminate lexical analysis;
End;

```

#### 4. Specification of Token. And Operations on Language.

Terms for a part of string

| Term | Definition |
|------|------------|
|------|------------|

|                                          |                                                                                                                              |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Prefix of S                              | A string obtained by removing zero or more trailing symbol of string S.<br>e.g., ban is prefix of banana.                    |
| Suffix of S                              | A string obtained by removing zero or more leading symbol of string S.<br>e.g., nana is suffix of banana.                    |
| Sub string of S                          | A string obtained by removing prefix and suffix from S.<br>e.g., nan is substring of banana.                                 |
| Proper prefix, suffix and substring of S | Any nonempty string x that is respectively prefix, suffix or substring of S, such that $s \neq x$                            |
| Subsequence of S                         | A string obtained by removing zero or more not necessarily contiguous symbol from S.<br>e.g., baaa is subsequence of banana. |

## Operation on languages

Definition of operation on language

| Operation                              | Definition                                                                |
|----------------------------------------|---------------------------------------------------------------------------|
| Union of L and M<br>Written L U M      | $L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$ |
| concatenation of L and M<br>Written LM | $LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$     |
| Kleene closure of L written $L^*$      | $L^*$ denotes "zero or more concatenation of" L.                          |
| Positive closure of L written $L^+$    | $L^+$ denotes "one or more concatenation of" L.                           |

## 5. Regular Expression

Regular Expressions are used to denote regular languages. An expression is regular if:

- $\phi$  is a regular expression for regular language  $\phi$ .
- $\epsilon$  is a regular expression for regular language  $\{\epsilon\}$ .

- If  $a \in \Sigma$  ( $\Sigma$  represents the input alphabet),  $a$  is regular expression with language  $\{a\}$ .
- If  $a$  and  $b$  are regular expression,  $a + b$  is also a regular expression with language  $\{a, b\}$ .
- If  $a$  and  $b$  are regular expression,  $ab$  (concatenation of  $a$  and  $b$ ) is also regular.
- If  $a$  is regular expression,  $a^*$  (0 or more times  $a$ ) is also regular.

**Regular Grammar :** A grammar is regular if it has rules of form  $A \rightarrow a$  or  $A \rightarrow aB$  or  $A \rightarrow \epsilon$  where  $\epsilon$  is a special symbol called NULL.

**Regular Languages :** A language is regular if it can be expressed in terms of regular expression.

### Closure Properties of Regular Languages

**Union :** If  $L_1$  and  $L_2$  are two regular languages, their union  $L_1 \cup L_2$  will also be regular. For example,  $L_1 = \{a^n \mid n \geq 0\}$  and  $L_2 = \{b^n \mid n \geq 0\}$

$L_3 = L_1 \cup L_2 = \{a^n \cup b^n \mid n \geq 0\}$  is also regular.

**Intersection :** If  $L_1$  and  $L_2$  are two regular languages, their intersection  $L_1 \cap L_2$  will also be regular. For example,

$L_1 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$  and  $L_2 = \{a^m b^n \cup b^n a^m \mid n \geq 0 \text{ and } m \geq 0\}$

$L_3 = L_1 \cap L_2 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$  is also regular.

**Concatenation :** If  $L_1$  and  $L_2$  are two regular languages, their concatenation  $L_1.L_2$  will also be regular. For example,

$L_1 = \{a^n \mid n \geq 0\}$  and  $L_2 = \{b^n \mid n \geq 0\}$

$L_3 = L_1.L_2 = \{a^m . b^n \mid m \geq 0 \text{ and } n \geq 0\}$  is also regular.

**Kleene Closure :** If  $L_1$  is a regular language, its Kleene closure  $L_1^*$  will also be regular. For example,

$L_1 = (a \cup b)$

$L_1^* = (a \cup b)^*$

**Complement :** If  $L(G)$  is regular language, its complement  $L'(G)$  will also be regular. Complement of a language can be found by subtracting strings which are in  $L(G)$  from all possible strings. For example,

$L(G) = \{a^n \mid n > 3\}$

$L'(G) = \{a^n \mid n \leq 3\}$

### Example 1:

Write the regular expression for the language accepting all combinations of a's, over the set  $\Sigma = \{a\}$

#### **Solution:**

All combinations of a's means a may be zero, single, double and so on. If a is appearing zero times, that means a null string. That is we expect the set of  $\{\epsilon, a, aa, aaa, \dots\}$ . So we give a regular expression for this as:

1.  $R = a^*$

That is Kleen closure of a.

### Example 2:

Write the regular expression for the language accepting all combinations of a's except the null string, over the set  $\Sigma = \{a\}$

#### **Solution:**

The regular expression has to be built for the language

1.  $L = \{a, aa, aaa, \dots\}$

This set indicates that there is no null string. So we can denote regular expression as:

$$R = a^+$$

### Example 3:

Write the regular expression for the language accepting all the string containing any number of a's and b's.

#### **Solution:**

The regular expression will be:

1.  $r.e. = (a + b)^*$

This will give the set as  $L = \{ \epsilon, a, aa, b, bb, ab, ba, aba, bab, \dots \}$ , any combination of a and b.

The  $(a + b)^*$  shows any combination with a and b even a null string. Example 2:

Write the regular expression for the language starting and ending with a and having any having any combination of b's in between.

**Solution:**

The regular expression will be:

1.  $R = a b^* a$

### Example 4:

Write the regular expression for the language starting with a but not having consecutive b's.

**Solution:** The regular expression has to be built for the language:

1.  $L = \{a, aba, aab, abaa, abab, \dots\}$

The regular expression for the above language is:

1.  $R = \{a + ab\}^*$

### Example 5:

Write the regular expression for the language accepting all the string in which any number of a's is followed by any number of b's is followed by any number of c's.

**Solution:** As we know, any number of a's means  $a^*$  any number of b's means  $b^*$ , any number of c's means  $c^*$ . Since as given in problem statement, b's appear after a's and c's appear after b's. So the regular expression could be:

1.  $R = a^* b^* c^*$

### Example 6:

Write the regular expression for the language over  $\Sigma = \{0\}$  having even length of the string.

**Solution:**

The regular expression has to be built for the language:

$$1. L = \{ \epsilon, 00, 0000, 000000, \dots \}$$

The regular expression for the above language is:

$$1. R = (00)^*$$

### Example 7:

Write the regular expression for the language having a string which should have atleast one 0 and atleast one 1.

**Solution:**

The regular expression will be:

$$1. R = [(0 + 1)^* 0 (0 + 1)^* 1 (0 + 1)^*] + [(0 + 1)^* 1 (0 + 1)^* 0 (0 + 1)^*]$$

### Example 8:

Describe the language denoted by following regular expression

$$1. r.e. = (b^* (aaa)^* b^*)^*$$

**Solution:**

The language can be predicted from the regular expression by finding the meaning of it. We will first split the regular expression as:

$$r.e. = (\text{any combination of } b\text{'s}) (aaa)^* (\text{any combination of } b\text{'s})$$

$L = \{\text{The language consists of the string in which } a\text{'s appear triples, there is no restriction on the number of } b\text{'s}\}$

### Example 9:

Write the regular expression for the language  $L$  over  $\Sigma = \{0, 1\}$  such that all the string do not contain the substring 01.

**Solution:**

The Language is as follows:

$$1. L = \{ \epsilon, 0, 1, 00, 11, 10, 100, \dots \}$$

The regular expression for the above language is as follows:

$$1. R = (1^* 0^*)$$

### Example 10:

Write the regular expression for the language containing the string over  $\{0, 1\}$  in which there are at least two occurrences of 1's between any two occurrences of 1's between any two occurrences of 0's.

**Solution:** At least two 1's between two occurrences of 0's can be denoted by  $(0111^*0)^*$ .

Similarly, if there is no occurrence of 0's, then any number of 1's are also allowed. Hence the r.e. for required language is:

$$1. R = (1 + (0111^*0))^*$$

### Example 11:

Write the regular expression for the language containing the string in which every 0 is immediately followed by 11.

**Solution:**

The regular expectation will be:

$$1. R = (011 + 1)^*$$

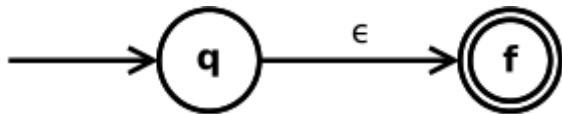
## 6. Finite Automata from Regular Expressions.(Thompson's Construction Method)

The algorithm works recursively by splitting an expression into its constituent subexpressions, from which the NFA will be constructed using a set of rules. More precisely, from a regular expression  $E$ , the obtained automaton  $A$  with the transition function  $\delta$  respects the properties

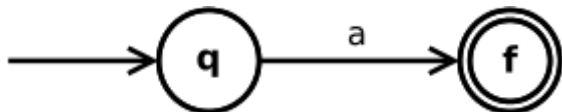
## Rules

The following rules are depicted according to Aho et al. (2007), [1] p. 122. In what follows,  $N(s)$  and  $N(t)$  are the NFA of the subexpressions  $s$  and  $t$ , respectively.

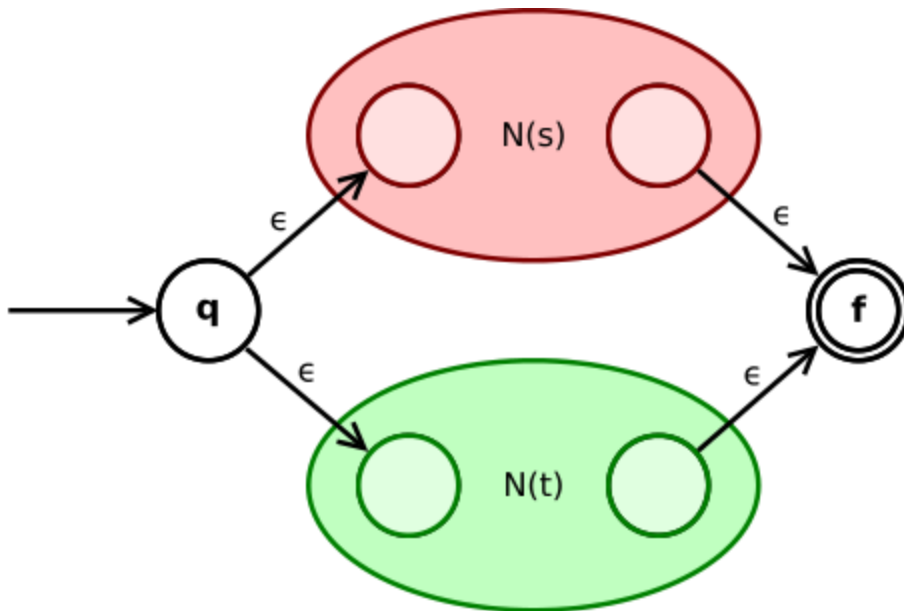
The empty-expression  $\epsilon$  is converted to



A symbol  $a$  of the input alphabet is converted to

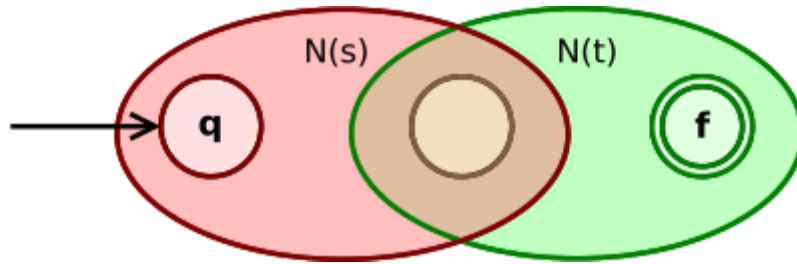


The union expression  $s|t$  is converted to



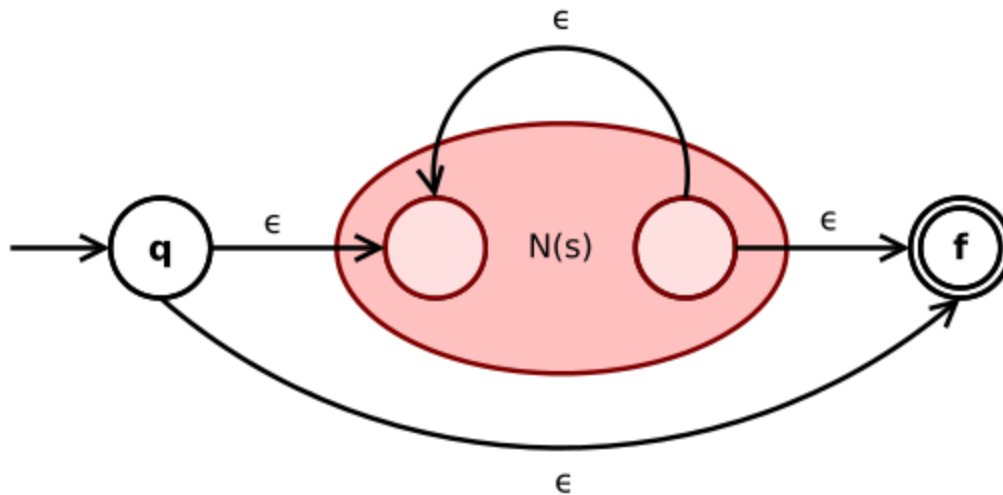
State  $q$  goes via  $\epsilon$  either to the initial state of  $N(s)$  or  $N(t)$ . Their final states become intermediate states of the whole NFA and merge via two  $\epsilon$ -transitions into the final state of the NFA.

The concatenation expression  $st$  is converted to



The initial state of  $N(s)$  is the initial state of the whole NFA. The final state of  $N(s)$  becomes the initial state of  $N(t)$ . The final state of  $N(t)$  is the final state of the whole NFA.

The Kleene star expression  $s^*$  is converted to

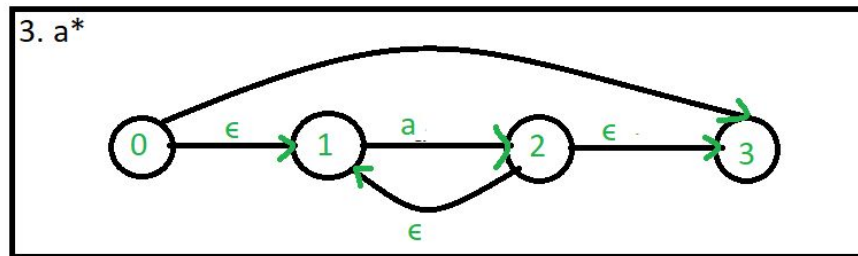
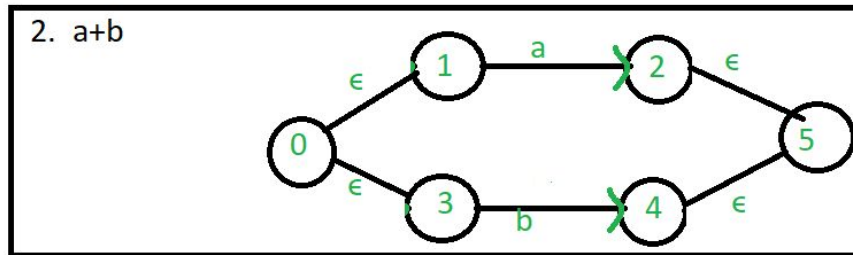
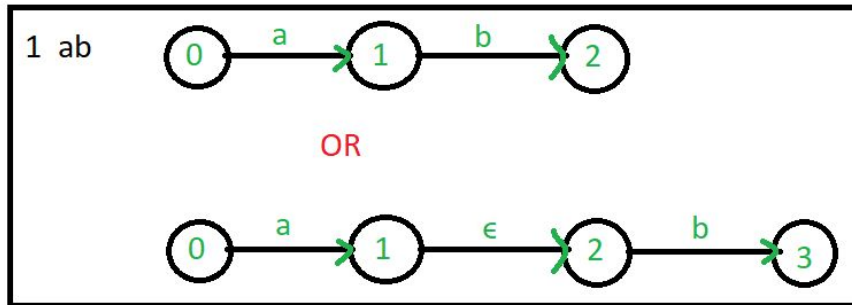


An  $\epsilon$ -transition connects initial and final state of the NFA with the sub-NFA  $N(s)$  in between. Another  $\epsilon$ -transition from the inner final to the inner initial state of  $N(s)$  allows for repetition of expression  $s$  according to the star operator.

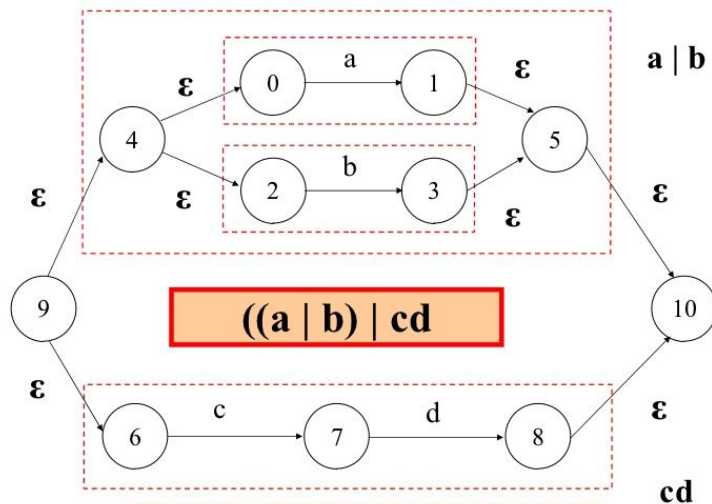
The parenthesized expression  $(s)$  is converted to  $N(s)$  itself.

## 7. Finite Automata from Regular Expressions.(Thompson's Construction Method)-

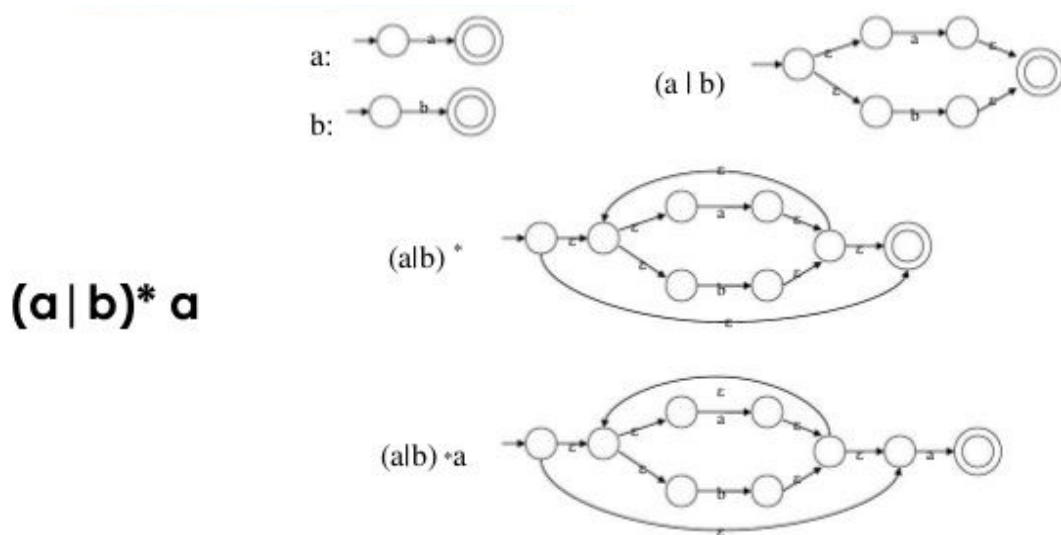
## Example 1



## Example 2



### Example 3



## 8. Finite Automata from Regular Expressions(FIRSTPOS, LASTPOS, FOLLOWPOS METHOD)

- Rules Table To find FIRSTPOS AND LASTPOS

| $n$                                                     | $nullable(n)$                       | $firstpos(n)$                                                                       | $lastpos(n)$                                                                     |
|---------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| leaf labeled $\epsilon$                                 | true                                | $\Phi$                                                                              | $\Phi$                                                                           |
| leaf labeled with position $i$                          | false                               | $\{i\}$                                                                             | $\{i\}$                                                                          |
| $\begin{array}{c}   \\ c_1 \quad c_2 \end{array}$       | $nullable(c_1)$ or $nullable(c_2)$  | $firstpos(c_1) \quad firstpos(c_2)$                                                 | $lastpos(c_1) \quad lastpos(c_2)$                                                |
| $\begin{array}{c} \bullet \\ c_1 \quad c_2 \end{array}$ | $nullable(c_1)$ and $nullable(c_2)$ | if $(nullable(c_1))$<br>$firstpos(c_1) \quad firstpos(c_2)$<br>else $firstpos(c_1)$ | if $(nullable(c_2))$<br>$lastpos(c_1) \quad lastpos(c_2)$<br>else $lastpos(c_2)$ |
| $\begin{array}{c} \circ \\   \\ c_1 \end{array}$        | true                                | $firstpos(c_1)$                                                                     | $lastpos(c_1)$                                                                   |

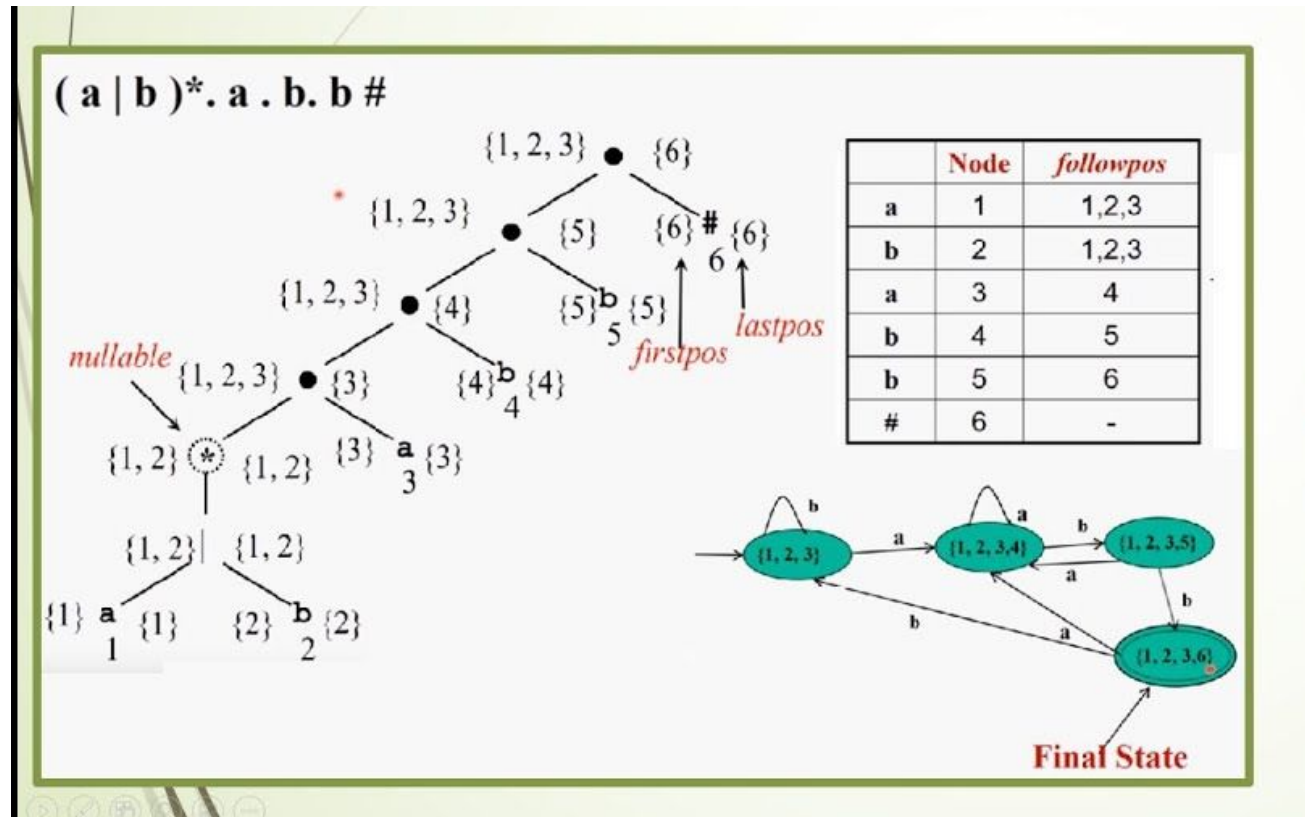
- Algorithm To find Followpos

followpos() is computed only for the leaf nodes that are labeled with the input symbols that constitute the language of the regular expression. followpos( $i$ ) is defined

as the set of positions that can follow position "i" in the syntax tree. Hence, this is an important function to construct the DFA. To compute followpos(i), the firstpos() and the lastpos() of all the nodes are necessary. The algorithm to compute followpos() is given in algorithm

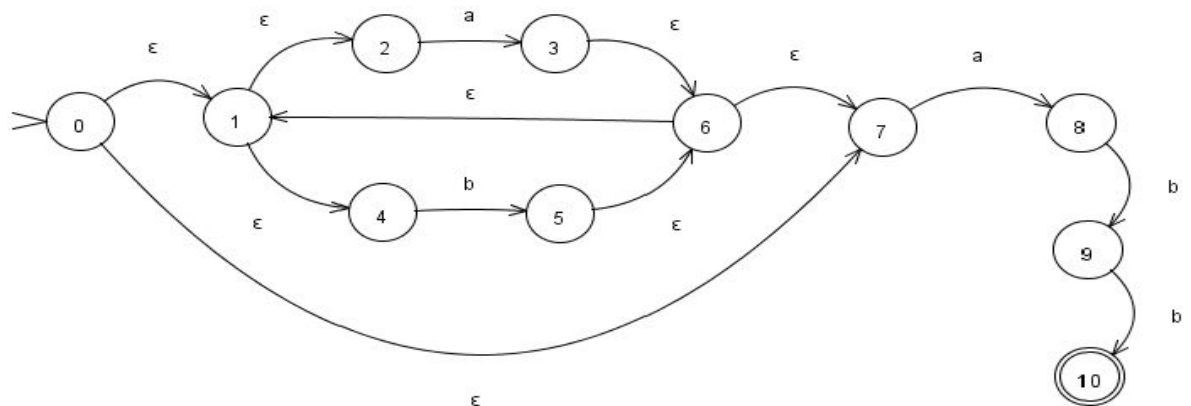
```
1. for each node n in the tree do
2.   if n is a concatenation node with left child c1 and right child c2 then
3.     for each i in lastpos(c1) do
4.       followpos(i) := followpos(i) U firstpos(c2)
     end do
5.   else if n is a *-node
6.     for each i in lastpos(n) do
7.       followpos(i) := followpos(i) U firstpos(n)
     end do
   end if
```

## **Example 1**



## 9. Conversion from NFA to DFA using Thompson's rule.

Ex:1  $(a+b)^*abb$



- $\epsilon$  - closure (0) = {0,1,2,4,7}    Let A
- Move(A,a) = {3,8}
- $\epsilon$  - closure (Move(A,a)) = {1,2,3,4,6,7,8}    Let B
- Move(A,b) = {5}

$\varepsilon$  – closure (Move(A,b)) = {1,2,4,5,6,7}    Let C

- Move(B,a) = {3,8}  
 $\varepsilon$  – closure (Move(B,a)) = {1,2,3,4,6,7,8}    B  
 Move(B,b) = {5,9}  
 $\varepsilon$  – closure (Move(B,b)) = {1,2,4,5,6,7,9}    Let D
- Move(C,a) = {3,8}  
 $\varepsilon$  – closure (Move(C,a)) = {1,2,3,4,6,7,8}    B  
 Move(C,b) = {5}  
 $\varepsilon$  – closure (Move(C,b)) = {1,2,4,5,6,7}    C
- Move(D,a) = {3,8}  
 $\varepsilon$  – closure (Move(D,a)) = {1,2,3,4,6,7,8}    B  
 Move(D,b) = {5,10}  
 $\varepsilon$  – closure (Move(D,b)) = {1,2,4,5,6,7,10}    Let E
- Move(E,a) = {3,8}  
 $\varepsilon$  – closure (Move(E,a)) = {1,2,3,4,6,7,8}    B  
 Move(E,b) = {5}  
 $\varepsilon$  – closure (Move(E,b)) = {1,2,4,5,6,7}    C

| States | a | b |
|--------|---|---|
| A      | B | C |
| B      | B | D |
| C      | B | C |
| D      | B | E |
| E      | B | C |

Table Transition table for (a+b)\*abb

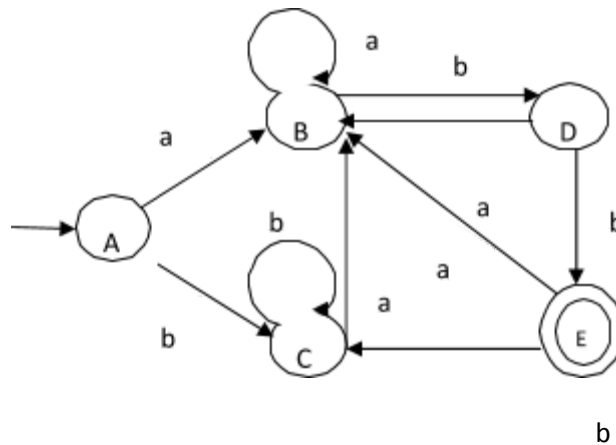


Fig.. DFA for (a+b)\*abb

## 10. DFA Optimization

To optimize the DFA you have to follow the various steps. These are as follows:

**Step 1:** Remove all the states that are unreachable from the initial state via any set of the transition of DFA.

**Step 2:** Draw the transition table for all pair of states.

**Step 3:** Now split the transition table into two tables T1 and T2. T1 contains all final states and T2 contains non-final states.

**Step 4:** Find the similar rows from T1 such that:

1.  $\delta(q, a) = p$
2.  $\delta(r, a) = p$

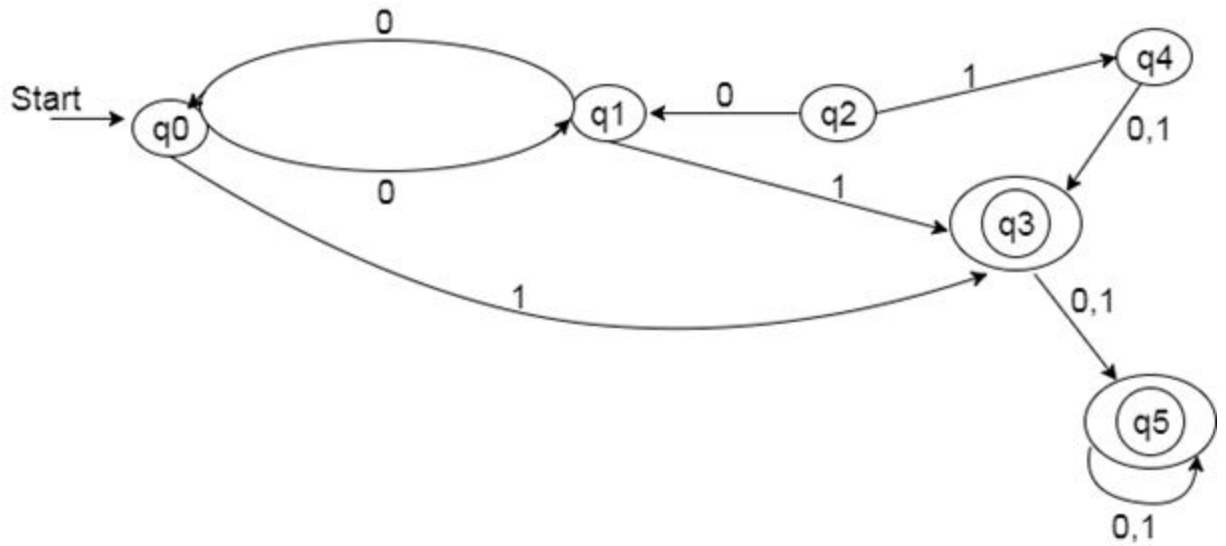
That means, find the two states which have same value of a and b and remove one of them.

**Step 5:** Repeat step 3 until there is no similar rows are available in the transition table T1.

**Step 6:** Repeat step 3 and step 4 for table T2 also.

**Step 7:** Now combine the reduced T1 and T2 tables. The combined transition table is the transition table of minimized DFA.

Example



Solution:

**Step 1:** In the given DFA, q2 and q4 are the unreachable states so remove them.

**Step 2:** Draw the transition table for rest of the states.

| State | 0  | 1  |
|-------|----|----|
| → q0  | q1 | q3 |
| q1    | q0 | q3 |
| *q3   | q5 | q5 |
| *q5   | q5 | q5 |

**Step 3:**

Now divide rows of transition table into two sets as:

1. One set contains those rows, which start from non-final states:

| State | 0  | 1  |
|-------|----|----|
| q0    | q1 | q3 |
| q1    | q0 | q3 |

2. Other set contains those rows, which starts from final states.

| State    | 0        | 1        |
|----------|----------|----------|
| q3<br>q5 | q5<br>q5 | q5<br>q5 |

**Step 4:** Set 1 has no similar rows so set 1 will be the same.

**Step 5:** In set 2, row 1 and row 2 are similar since q3 and q5 transit to same state on 0 and 1. So skip q5 and then replace q5 by q3 in the rest.

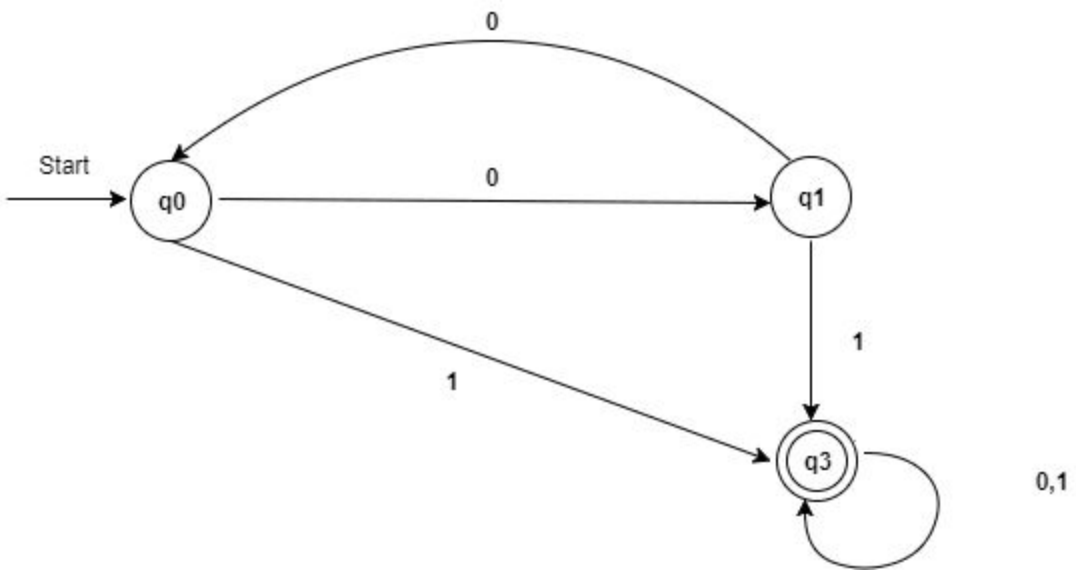
| State | 0  | 1  |
|-------|----|----|
| q3    | q3 | q3 |

**Step 6:** Now combine set 1 and set 2 as:

| State            | 0              | 1              |
|------------------|----------------|----------------|
| →q0<br>q1<br>*q3 | q1<br>q0<br>q3 | q3<br>q3<br>q3 |

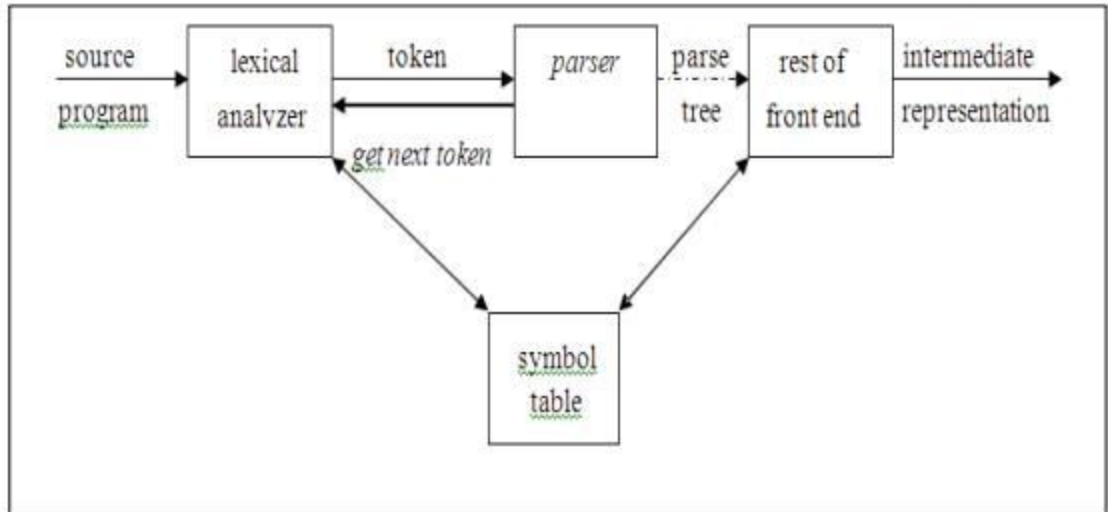
Now it is the transition table of minimized DFA.

Transition diagram of minimized DFA:



## Unit 3: Parsing Theory

### 1. Role of Parser



- In our compiler model, the parser obtains a string of tokens from lexical analyzer, as shown in fig.
- We expect the parser to report any syntax error. It should commonly occurring errors.
- The methods commonly used for parsing are classified as a top down or bottom up parsing.
- In top down parsing parser, build parse tree from top to bottom, while bottom up parser starts from leaves and work up to the root.
- In both the cases, the input to the parser is scanned from left to right, one symbol at a time.
- We assume the output of parser is some representation of the parse tree for the stream of tokens produced by the lexical analyzer.

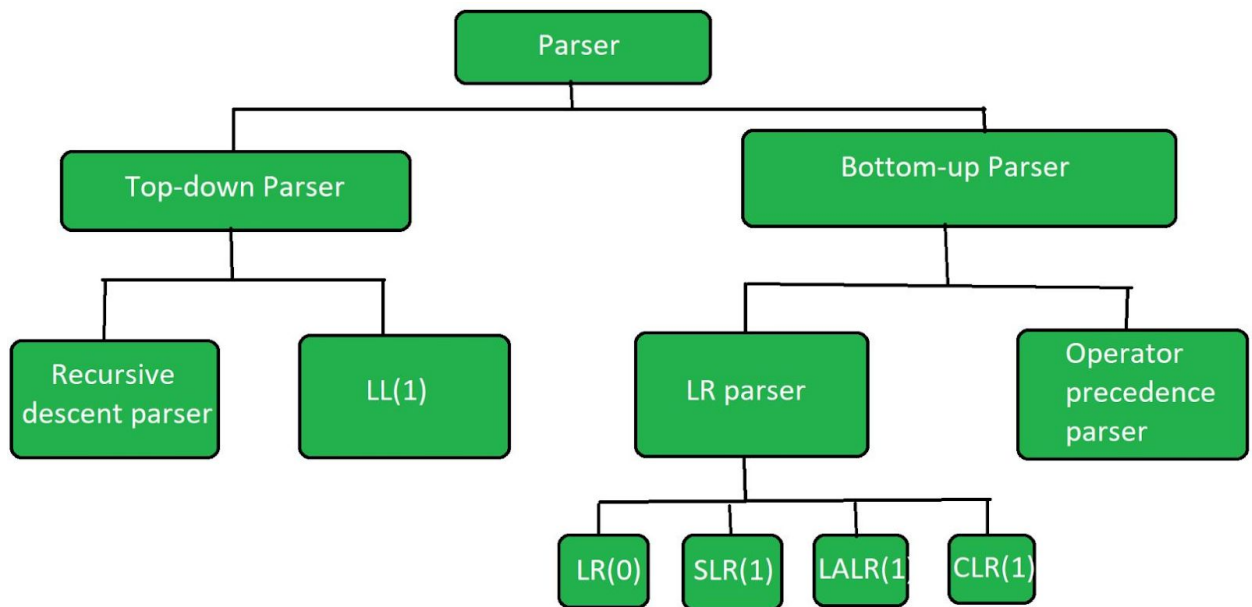
### 2. Types of Parsing.

Parsing or syntactic analysis is the process of analyzing a string of symbols according to the rules of a formal grammar.

- a. Parsing is a technique that takes input string and produces output either a parse tree if string is valid sentence of grammar, or an error message indicating that string is not a valid sentence of given grammar. Types of parsing are,

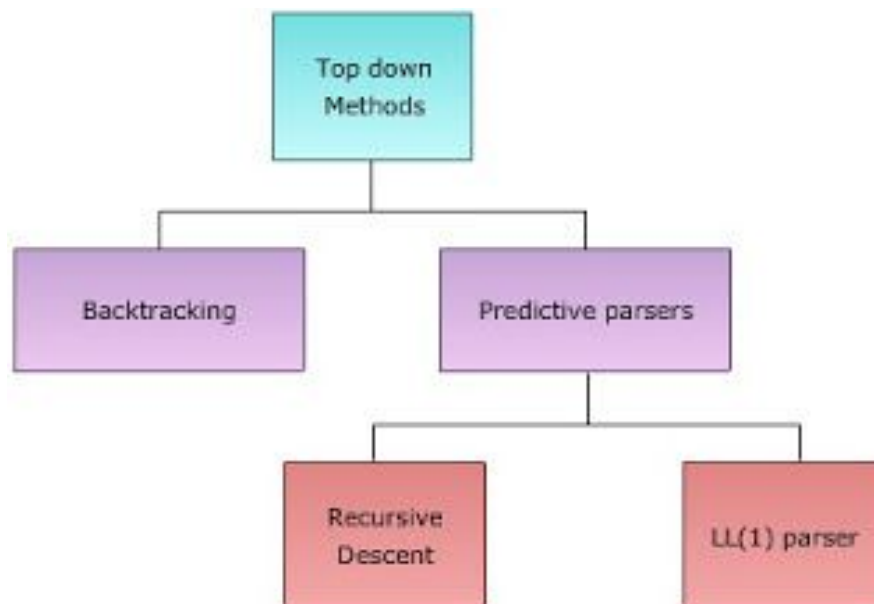
**Top down parsing:** In top down parsing parser build parse tree from top to bottom.

**Bottom up parsing:** While bottom up parser starts from leaves and work up to the root.



### 3. Top Down Parser

The top-down parsing technique parses the input, and starts constructing a parse tree from the root node gradually moving down to the leaf nodes. The types of top-down parsing are depicted below:



## 4. Problems with Top-Down parsing.

### 4.1.Backtracking

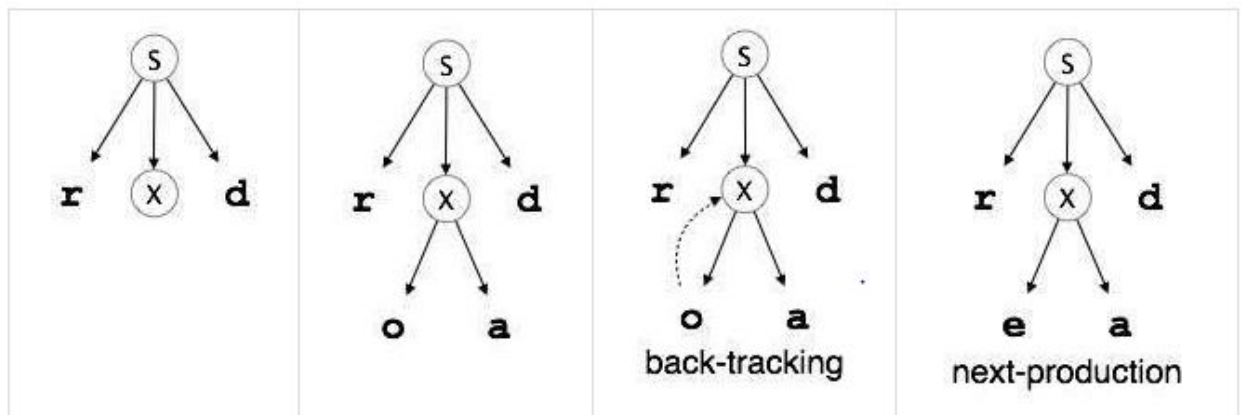
Top- down parsers start from the root node (start symbol) and match the input string against the production rules to replace them (if matched). To understand this, take the following example of CFG:

$$S \rightarrow rXd \mid rZd$$
$$X \rightarrow oa \mid ea$$
$$Z \rightarrow ai$$

For an input string: read, a top-down parser, will behave like this:

It will start with S from the production rules and will match its yield to the left-most letter of the input, i.e. 'r'. The very production of S ( $S \rightarrow rXd$ ) matches with it. So the top-down parser advances to the next input letter (i.e. 'e'). The parser tries to expand non-terminal 'X' and checks its production from the left ( $X \rightarrow oa$ ). It does not match with the next input symbol. So the top-down parser backtracks to obtain the next production rule of X, ( $X \rightarrow ea$ ).

Now the parser matches all the input letters in an ordered manner. The string is accepted.



### 4.2. Left Recursion

Grammar of the form,

$$S \rightarrow S / a / b$$

It is called left recursive where S is any non Terminal and a, and b are any set of terminals.

Problem with Left Recursion:

If a left recursion is present in any grammar then, during parsing in the the syntax analysis part of compilation there is a chance that the grammar will create infinite loop. This is because at every time of production of grammar S will produce another S without checking any condition.

Algorithm to Remove Left Recursion with an example:

Suppose we have a grammar which contains left recursion:

$$S \rightarrow S a / S b / c / d$$

1. Check if the given grammar contains left recursion, if present then separate the production and start working on it.

In our example,

$$S \rightarrow S a / S b / c / d$$

2. Introduce a new nonterminal and write it at the last of every terminal. We produce a new nonterminal S' and write new production as,

$$S \rightarrow cS' / dS'$$

Write newly produced nonterminal in LHS and in RHS it can either produce or it can produce new production in which the terminals or non terminals which followed the previous LHS will be replaced by new nonterminal at last.

$$S' \rightarrow ? / aS' / bS'$$

So after conversion the new equivalent production is  
 $S \rightarrow cS' / dS'$

3.  $S' \rightarrow ? / aS' / bS'$

Example 1:

$$S \rightarrow \overset{\alpha_1}{S} \overset{\alpha_2}{a} \overset{\alpha_3}{b} / \overset{\alpha_2}{S} \overset{\alpha_3}{c} \overset{\alpha_1}{d} / \overset{\alpha_3}{S} \overset{\beta_1}{e} \overset{\beta_2}{f} / \overset{\beta_1}{g} / \overset{\beta_2}{h}$$

$$S \rightarrow gS' / hS'$$

$$S' \rightarrow \epsilon / abS' / cdS' / efS'$$

Example 2 :

$$S \rightarrow (L)/a \quad \text{No left Recursion}$$

$$L \rightarrow L, S/S \quad \text{left Recursion}$$

$$L \rightarrow Sl'$$

$$L' \rightarrow \epsilon / SL'$$

### Example 3:

$$\begin{array}{ccccccc} E & \rightarrow & E & + & T & / & T \\ \underbrace{\phantom{E}} & & \underbrace{\phantom{E}} & & \underbrace{\phantom{T}} & & \underbrace{\phantom{T}} \\ A & & A & & \alpha & & \beta \end{array}$$

$$E \rightarrow TE'$$

$$E' \rightarrow +T E' / \epsilon$$

$$A \rightarrow A\alpha / \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \epsilon$$

## 4.3. Left Factoring

Removing Left Factoring :

A grammar is said to be left factored when it is of the form –

$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \alpha \beta_3 \mid \dots \mid \alpha \beta_n \mid \gamma$  i.e the productions start with the same terminal (or set of terminals). On seeing the input  $\alpha$  we cannot immediately tell which production to choose to expand A.

Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive or top down parsing. When the choice between two alternative A-productions is not clear, we may be able to rewrite the productions to defer the decision until enough of the input has been seen to make the right choice.

For the grammar  $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \alpha \beta_3 \mid \dots \mid \alpha \beta_n \mid \gamma$

The equivalent left factored grammar will be –

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$$

Example 1:

$$S \rightarrow iEtS \mid iEtSeS/a/b$$

$$S \rightarrow iEtSS'/a/b$$

$$S' \rightarrow eS/\epsilon$$

Example 2:

$$S \rightarrow a/ab/abc/abcd/e/f$$

$$S \rightarrow aS'/e/f$$

$$S' \rightarrow bS''/\epsilon \quad \text{— for single } a$$

$$S'' \rightarrow cS'''/\epsilon \quad \text{— for } ab$$

$$S''' \rightarrow d/\epsilon \quad \text{— for } abc$$

## 4.4. Ambiguous Grammar

Context Free Grammars (CFGs) are classified based on:

- Number of Derivation trees
- Number of strings

Depending on Number of Derivation trees, CFGs are subdivided into 2 types:

- Ambiguous grammars

- Unambiguous grammars

### **Ambiguous grammar:**

A CFG is said to be ambiguous if there exists more than one derivation tree for the given input string i.e., more than one **LeftMost Derivation Tree (LMDT)** or **RightMost Derivation Tree (RMDT)**.

**Definition:**  $G = (V, T, P, S)$  is a CFG is said to be ambiguous if and only if there exist a string in  $T^*$  that has more than one parse tree.

where  $V$  is a finite set of variables.

$T$  is a finite set of terminals.

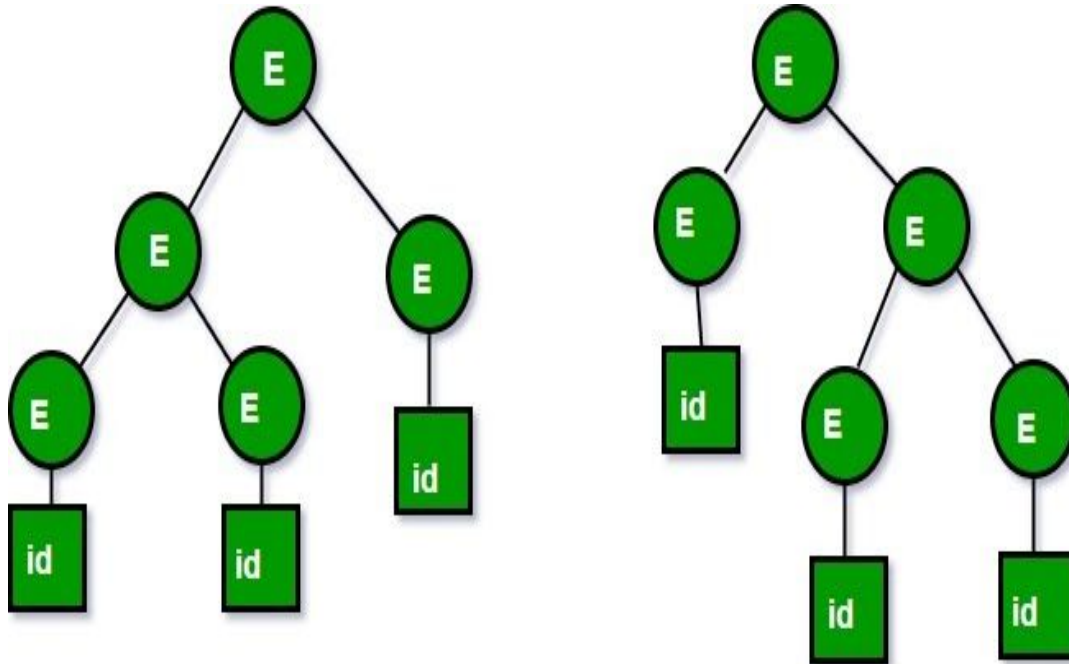
$P$  is a finite set of productions of the form,  $A \rightarrow \alpha$ , where  $A$  is a variable and  $\alpha \in (V \cup T)^*$   $S$  is a designated variable called the start symbol.

### **For Example:**

1. Let us consider this grammar :  $E \rightarrow E+E \mid id$

We can create 2 parse tree from this grammar to obtain a string **id+id+id** :

The following are the 2 parse trees generated by left most derivation:



Both the above parse trees are derived from same grammar rules but both parse trees are different. Hence the grammar is ambiguous.

2. Let us now consider the following grammar:

Set of alphabets  $\Sigma = \{0, \dots, 9, +, *, (, )\}$

$E \rightarrow I$

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$I \rightarrow \varepsilon \mid 0 \mid 1 \mid \dots \mid 9$

From the above grammar String **3\*2+5** can be derived in 2 ways:

I) First leftmost derivation

$E \Rightarrow E * E$

$\Rightarrow I * E$

$\Rightarrow 3 * E + E$

$\Rightarrow 3 * I + E$

$\Rightarrow 3 * 2 + E$

$\Rightarrow 3 * 2 + I$

$\Rightarrow 3 * 2 + 5$

II) Second leftmost derivation

$E \Rightarrow E + E$

$\Rightarrow E * E + E$

$\Rightarrow I * E + E$

$\Rightarrow 3 * E + E$

$\Rightarrow 3 * I + E$

$\Rightarrow 3 * 2 + I$

$\Rightarrow 3 * 2 + 5$

Following are some examples of ambiguous grammars:

- $S \rightarrow aS \mid Sa \mid \epsilon$
- $E \rightarrow E + E \mid E * E \mid id$
- $A \rightarrow AA \mid (A) \mid a$

- $S \rightarrow SS \mid AB$ ,  $A \rightarrow Aa \mid a$ ,  $B \rightarrow Bb \mid b$

Whereas following grammars are unambiguous:

- $S \rightarrow (L) \mid a$ ,  $L \rightarrow LS \mid S$
- $S \rightarrow AA$ ,  $A \rightarrow aA$ ,  $A \rightarrow b$

## 5. Recursive Descent Parser

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right. It uses procedures for every terminal and non-terminal entity. This parsing technique recursively parses the input to make a parse tree, which may or may not require back-tracking. But the grammar associated with it (if not left factored) cannot avoid back-tracking. A form of recursive-descent parsing that does not require any back-tracking is known as predictive parsing.

This parsing technique is regarded recursively as it uses context-free grammar which is recursive in nature.

**Example:**

BEFORE REMOVING LEFT RECURSION

AFTER REMOVING LEFT RECURSION

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow ( E ) \mid id$

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid e$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid e$

---

$$F \rightarrow ( E ) \mid id$$

**\*\*Here e is Epsilon**

**For Recursive Descent Parser, we are going to write one program for every variable.**

Example:

Grammar:  $E \rightarrow i E'$

$E' \rightarrow + i E' \mid e$

```
int main()
{
    // E is a start symbol.
    E();

    // if lookahead = $, it represents the end of the string
    // Here l is lookahead.
    if (l == '$')
        printf("Parsing Successful");
}
```

```
// Definition of E, as per the given production
E()
{
    if (l == 'i') {
        match('i');
        E'();
    }
}
```

```
// Definition of E' as per the given production
E'()
{
    if (l == '+') {
        match('+');
        match('i');
        E'();
    }
}
```

```

    }
    else
        return ();
}

// Match function
match(char t)
{
    if (l == t) {
        l = getchar();
    }
    else
        printf("Error");
}

```

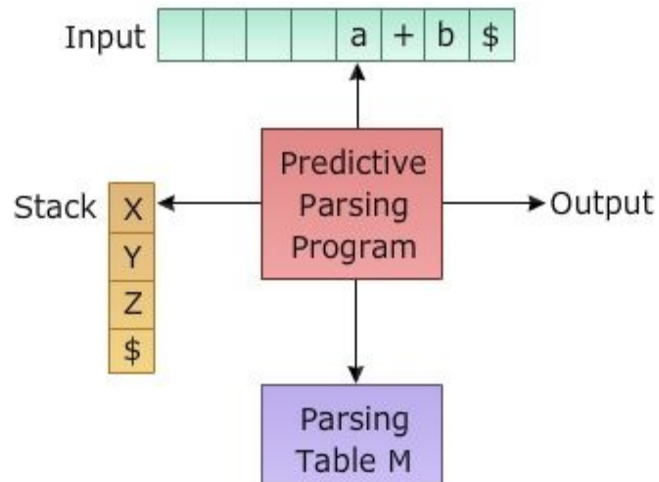
## 6. LL(1) Parser OR Predictive parsing

- This top-down parsing is non-recursive. LL (1) – the first L indicates input is scanned from left to right. The second L means it uses leftmost derivation for input string and 1 means it uses only input symbol to predict the parsing process.
- The block diagram for LL(1) parser is given below,
- The data structure used by LL(1) parser are input buffer, stack and parsing table.
- The parser works as follows,
- The parsing program reads top of the stack and a current input symbol. With the help of these two symbols parsing action can be determined.
- The parser consult the table  $M[A, a]$  each time while taking the parsing actions hence this type of parsing method is also called table driven parsing method.
- The input is successfully parsed if the parser reaches the halting configuration. When the stack is empty and next token is \$ then it

corresponds to successful parsing.

Steps to construct LL(1) parser

1. Remove left recursion / Perform left factoring.
2. Compute FIRST and FOLLOW of nonterminals.
3. Construct predictive parsing table.
4. Parse the input string with the help of parsing table.



**Example:** $E \rightarrow E+T/T$  $T \rightarrow T*F/F$  $F \rightarrow (E)/id$ 

Step1: Remove left recursion

 $E \rightarrow TE'$  $E' \rightarrow +TE' \mid \epsilon$  $T \rightarrow FT'$  $T' \rightarrow *FT' \mid \epsilon$  $F \rightarrow (E) \mid id$ 

Step2: Compute FIRST &amp; FOLLOW

|    | FIRST            | FOLLOW     |
|----|------------------|------------|
| E  | {(,id}           | {\$,)}     |
| E' | {+, $\epsilon$ } | {\$,)}     |
| T  | {(,id}           | {+,\$,)}   |
| T' | {*, $\epsilon$ } | {+,\$,)}   |
| F  | {(,id}           | {*,+,\$,)} |

Table 3.1.3 first &amp; follow set

Step3: Predictive Parsing Table

|    | id                  | +                         | *                     | (                   | )                         | \$                        |
|----|---------------------|---------------------------|-----------------------|---------------------|---------------------------|---------------------------|
| E  | $E \rightarrow TE'$ |                           |                       | $E \rightarrow TE'$ |                           |                           |
| E' |                     | $E' \rightarrow +TE'$     |                       |                     | $E' \rightarrow \epsilon$ | $E' \rightarrow \epsilon$ |
| T  | $T \rightarrow FT'$ |                           |                       | $T \rightarrow FT'$ |                           |                           |
| T' |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |
| F  | $F \rightarrow id$  |                           |                       | $F \rightarrow (E)$ |                           |                           |

Table 3.1.4 predictive parsing table

Step4: Parse the string

| Stack    | Input      | Action                    |
|----------|------------|---------------------------|
| \$E      | id+id*id\$ |                           |
| \$E'T    | id+id*id\$ | $E \rightarrow TE'$       |
| \$E'T'F  | id+id*id\$ | $T \rightarrow FT'$       |
| \$E'T'id | id+id*id\$ | $F \rightarrow id$        |
| \$E'T'   | +id*id\$   |                           |
| \$E'     | +id*id\$   | $T' \rightarrow \epsilon$ |

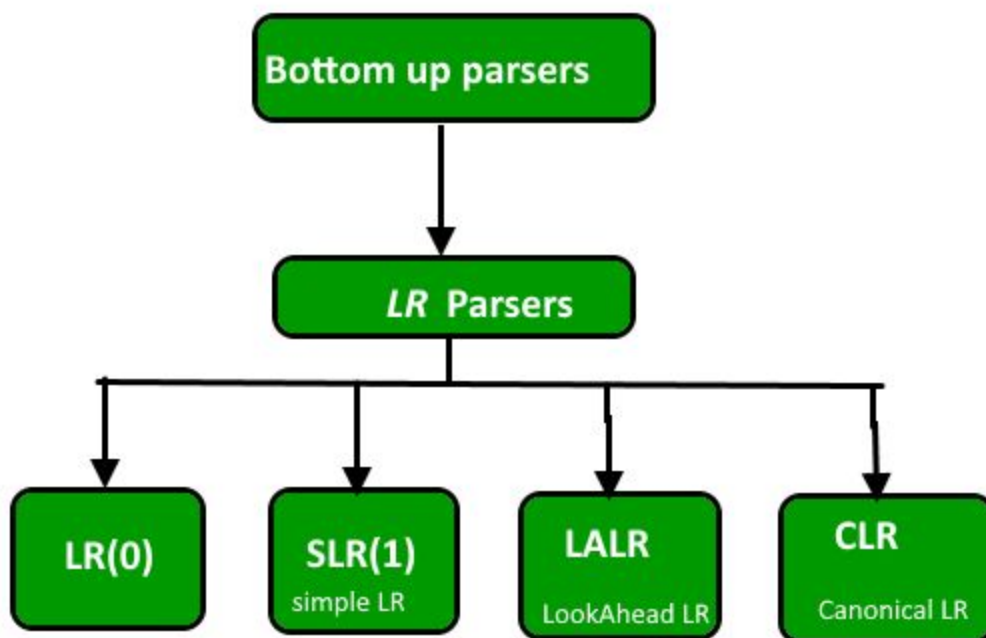
|           |          |         |
|-----------|----------|---------|
| \$ E'T+   | +id*id\$ | E'□+TE' |
| \$ E'T    | id*id\$  |         |
| \$ E'T'F  | id*id\$  | T□FT'   |
| \$ E'T'id | id*id\$  | F□id    |
| \$ E'T'   | *id\$    |         |
| \$ E'T'F* | *id\$    | T'□*FT' |
| \$ E'T'F  | id\$     |         |
| \$ E'T'id | id\$     | F□id    |

|           |      |                           |
|-----------|------|---------------------------|
| $\$ E'T'$ | $\$$ |                           |
| $\$ E'$   | $\$$ | $T' \rightarrow \epsilon$ |
| $\$$      | $\$$ | $E' \rightarrow \epsilon$ |

Table 3.1.5. moves made by predictive parse

## 7. Bottom Up Parsing

Bottom-up parsing starts from the leaf nodes of a tree and works in upward direction till it reaches the root node. Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol. The image given below depicts the bottom-up parsers available.



## 8. Shift Reduce Parser

**Shift Reduce parser** attempts for the construction of parse in a similar manner as done in bottom up parsing i.e. the parse tree is constructed from leaves(bottom) to the root(up). A more general form of shift reduce parser is LR parser.

This parser requires some data structures i.e.

- A input buffer for storing the input string.
- A stack for storing and accessing the production rules.

### Basic Operations –

- **Shift:** This involves moving of symbols from input buffer onto the stack.
- **Reduce:** If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of production rule is popped out of stack and LHS of production rule is pushed onto the stack.
- **Accept:** If only start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accept action is obtained, it means successful parsing is done.
- **Error:** This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

**Example 1** – Consider the grammar

$$S \rightarrow S + S$$

$$S \rightarrow S * S$$

$$S \rightarrow id$$

Perform Shift Reduce parsing for input string “id + id + id”.

| Stack    | Input Buffer | Parsing Action                |
|----------|--------------|-------------------------------|
| \$       | id+id+id\$   | Shift                         |
| \$id     | +id+id\$     | Reduce by $S \rightarrow id$  |
| \$S      | +id+id\$     | Shift                         |
| \$S+     | id+id\$      | Shift                         |
| \$S+id   | +id\$        | Reduce by $S \rightarrow id$  |
| \$S+S    | +id\$        | Shift                         |
| \$S+S+   | id\$         | Shift                         |
| \$S+S+id | \$           | Reduce by $S \rightarrow id$  |
| \$S+S+S  | \$           | Reduce by $S \rightarrow S+S$ |
| \$S+S    | \$           | Reduce by $S \rightarrow S+S$ |
| \$S      | \$           | Accept                        |

**Example 2** – Consider the grammar

$E \rightarrow 2E2$

$E \rightarrow 3E3$

$E \rightarrow 4$

Perform Shift Reduce parsing for input string “32423”.

| Stack  | Input Buffer | Parsing Action                |
|--------|--------------|-------------------------------|
| \$     | 32423\$      | Shift                         |
| \$3    | 2423\$       | Shift                         |
| \$32   | 423\$        | Shift                         |
| \$324  | 23\$         | Reduce by $E \rightarrow 4$   |
| \$32E  | 23\$         | Shift                         |
| \$32E2 | 3\$          | Reduce by $E \rightarrow 2E2$ |
| \$3E   | 3\$          | Shift                         |
| \$3E3  | \$           | Reduce by $E \rightarrow 3E3$ |
| \$E    | \$           | Accept                        |

## 9. Operator Precedence Grammar

A grammar that is used to define mathematical operators is called an operator grammar or operator precedence grammar. Such grammars have the restriction that no production has either an empty right-hand side (null productions) or two adjacent non-terminals in its right-hand side.

Examples –

This is an example of operator grammar:

$$E \rightarrow E + E / E * E / id$$

However, the grammar given below is not an operator grammar because two non-terminals are adjacent to each other:

$$S \rightarrow SAS/a$$
$$A \rightarrow bSb/b$$

We can convert it into an operator grammar, though:

$$S \rightarrow SbSbS/SbS/a$$
$$A \rightarrow bSb/b$$

Operator precedence parser –

An operator precedence parser is a bottom-up parser that interprets an operator grammar. This parser is only used for operator grammars. *Ambiguous grammars are not allowed* in any parser except operator precedence parser.

There are two methods for determining what precedence relations should hold between a pair of terminals:

1. Use the conventional associativity and precedence of operator.
2. The second method of selecting operator-precedence relations is first to construct an unambiguous grammar for the language, a grammar that reflects the correct associativity and precedence in its parse trees.

This parser relies on the following three precedence relations:  $\lessdot, \doteq, \gtrdot$

$a \lessdot b$  This means a “yields precedence to” b.

$a \gtrdot b$  This means a “takes precedence over” b.

$a \doteq b$  This means a “has same precedence as” b.

|    | id      | +       | *       | \$      |
|----|---------|---------|---------|---------|
| id |         | $\succ$ | $\succ$ | $\succ$ |
| +  | $\prec$ | $\succ$ | $\prec$ | $\succ$ |
| *  | $\prec$ | $\succ$ | $\succ$ | $\succ$ |
| \$ | $\prec$ | $\prec$ | $\prec$ |         |

Figure – Operator precedence relation table for grammar  $E \rightarrow E + E / E * E / id$

There is not given any relation between id and id as id will not be compared and two variables can not come side by side. There is also a disadvantage of this table – if we have n operators then size of table will be  $n*n$  and complexity will be  $O(n^2)$ . In order to decrease the size of table, we use operator function table.

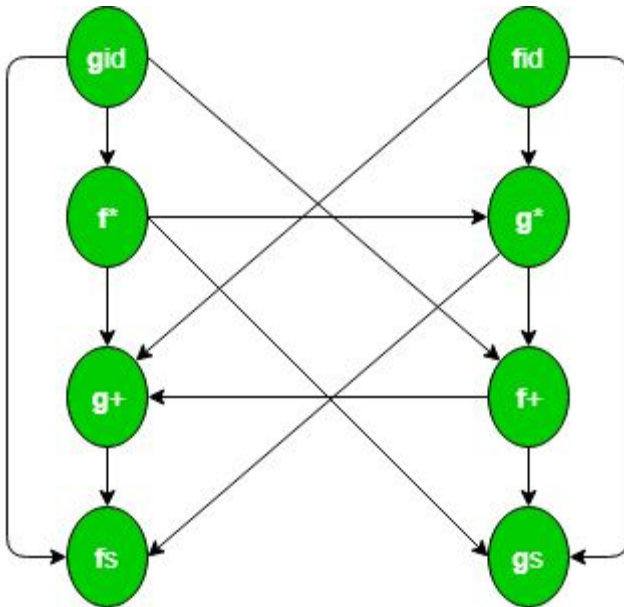
Operator precedence parsers usually do not store the precedence table with the relations; rather they are implemented in a special way. Operator precedence parsers use precedence functions that map terminal symbols to integers, and the precedence relations between the symbols are implemented by numerical comparison. The parsing table can be encoded by two precedence functions f and g that map terminal symbols to integers. We select f and g such that:

1.  $f(a) < g(b)$  whenever a yields precedence to b
2.  $f(a) = g(b)$  whenever a and b have the same precedence
3.  $f(a) > g(b)$  whenever a takes precedence over b

Example – Consider the following grammar:

$$E \rightarrow E + E / E * E / ( E ) / id$$

This is the directed graph representing the precedence function:



Since there is no cycle in the graph, we can make this function table:

|   | id | + | * | \$ |
|---|----|---|---|----|
| f | 4  | 2 | 4 | 0  |
| g | 5  | 1 | 3 | 0  |

fid  $\rightarrow$  g\*  $\rightarrow$  f+  $\rightarrow$  g+  $\rightarrow$  f\$

gid  $\rightarrow$  f\*  $\rightarrow$  g\*  $\rightarrow$  f+  $\rightarrow$  g+  $\rightarrow$  f\$

Size of the table is  $2n$ .

## 10. LR Parser

- LR parsing is most efficient method of bottom up parsing which can be used to parse large class of context free grammar.
- The technique is called LR(k) parsing; the "L" is for left to right scanning of input symbol, the "R" for constructing right most derivation in reverse, and the k for the number of input symbols of lookahead that

are used in making parsing decision.

- There are three types of LR parsing,
  - SLR (Simple LR)
  - LR(O)
  - CLR (Canonical LR)
  - LALR (Lookahead LR)
- The schematic form of LR parser is given in figure 3.1.6.
- The structure of input buffer for storing the input string, a stack for storing a grammar symbols, output and a parsing table comprised of two parts, namely action and goto.
  - **Properties of LR parser**
- LR parser can be constructed to recognize most of the programming language for which CFG can be written.
- The class of grammars that can be parsed by LR parser is a superset of class of grammars that can be parsed using predictive parsers.
- LR parser works using non back tracking shift reduce technique.
- LR parser can detect a syntactic error as soon as possible.

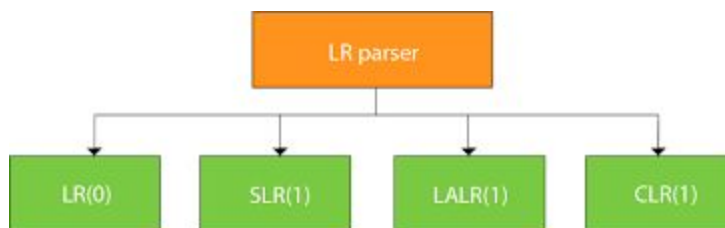
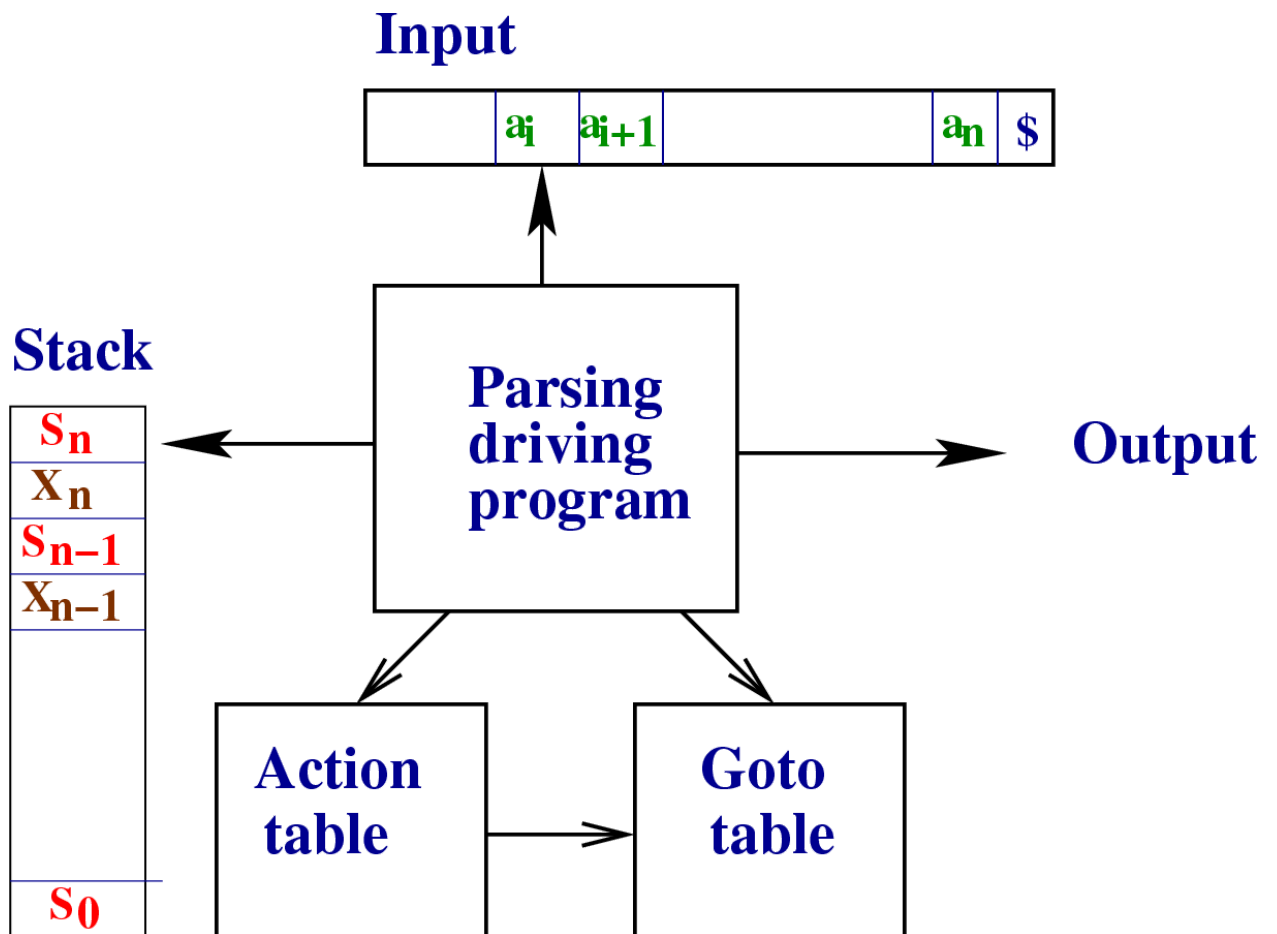


Fig: Types of LR parser



## 11. LR(0) Parser with Example.

LR(0) Parser We need two functions –

Closure()

Goto()

Augmented Grammar

If  $G$  is a grammar with start symbol  $S$  then  $G'$ , the augmented grammar for  $G$ , is the grammar with new start symbol  $S'$  and a production  $S' \rightarrow S$ . The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of input.

Let a grammar be  $S \rightarrow AA$

$A \rightarrow aA \mid b$

The augmented grammar for the above grammar will be

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

LR(0) Items

An LR(0) item of a grammar  $G$  is a production of  $G$  with a dot at some position in the right side.

$S \rightarrow ABC$  yields four items

$S \rightarrow .ABC$

$S \rightarrow A.BC$

$S \rightarrow AB.C$

$S \rightarrow ABC.$

The production  $A \rightarrow \epsilon$  generates only one item  $A \rightarrow .\epsilon$

Closure Operation:

If  $I$  is a set of items for a grammar  $G$ , then  $\text{closure}(I)$  is the set of items constructed from  $I$  by the two rules:

1. Initially every item in  $I$  is added to  $\text{closure}(I)$ .

2. If  $A \rightarrow \alpha . B \beta$  is in  $\text{closure}(I)$  and  $B \rightarrow \gamma$  is a production then add the item  $B \rightarrow . \gamma$  to  $I$ , If it is not already there. We apply this rule until no more items can be added to  $\text{closure}(I)$ .

Eg:

$S' \rightarrow S$   
 $S \rightarrow AA$   
 $A \rightarrow aA/b$

$\text{Closure}(S' \rightarrow S) = S' \rightarrow .S$  → non terminal. So add production of S  
 $S \rightarrow .A A$  → non terminal after dot. So add production of A  
 $A \rightarrow .aA / .b$   
 $\text{Closure}(S \rightarrow A.A) = S \rightarrow A.A$   
 $A \rightarrow .aA / .b$   
 $\text{Closure}(A \rightarrow aA) = A \rightarrow a.A$

Goto Operation :

Goto( $I, X$ ) = 1. Add  $I$  by moving dot after  $X$ .

2. Apply closure to first step.

$S' \rightarrow S$   
 $S \rightarrow AA$   
 $A \rightarrow aA/b$

$\text{Goto}(S' \rightarrow .S, S) = S' \rightarrow S. \square$  → nothing is present after dot, no closure  
 $S \rightarrow .A A$  → non terminal after dot. So add production of A  
 $A \rightarrow .aA / .b$

$\text{Goto}(S \rightarrow .AA, A) = S \rightarrow A.A$  → apply closure  
 $A \rightarrow .aA / .b$

$\text{Goto}(A \rightarrow .aA, a) = A \rightarrow a.A$  → apply closure  
 $A \rightarrow .aA / .b$

Construction of GOTO graph-

- State I0 – closure of augmented LR(0) item
- Using I0 find all collection of sets of LR(0) items with the help of DFA
- Convert DFA to LR(0) parsing table

Construction of LR(0) parsing table:

- The action function takes as arguments a state i and a terminal a (or \$ , the input end marker). The value of ACTION[i, a] can have one of four forms:
  1. Shift j, where j is a state.
  2. Reduce  $A \rightarrow \beta$ .
  3. Accept
  4. Error

- We extend the GOTO function, defined on sets of items, to states: if  $\text{GOTO}[I_i, A] = I_j$  then GOTO also maps a state  $i$  and a nonterminal  $A$  to state  $j$ .

Eg:

Consider the grammar  $S \rightarrow AA$

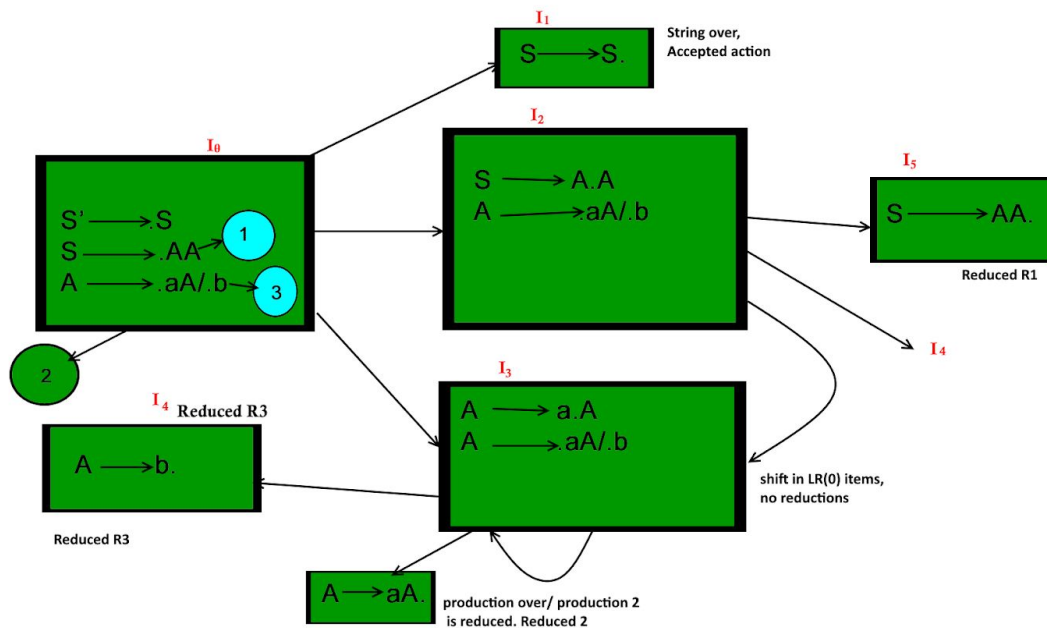
$A \rightarrow aA \mid b$

Augmented grammar  $S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

The LR(0) parsing table for above GOTO graph will be –



Action part of the table contains all the terminals of the grammar whereas the goto part contains all the nonterminals. For every state of goto graph we write all the goto operations in

the table. If goto is applied to a terminal than it is written in the action part if goto is applied on a nonterminal it is written in goto part. If on applying goto a production is reduced ( i.e if the dot reaches at the end of production and no further closure can be applied) then it is denoted as Ri and if the production is not reduced (shifted) it is denoted as Si.

If a production is reduced it is written under the terminals given by follow of the left side of the production which is reduced for ex: in I5 S->AA is reduced so R1 is written under the terminals in follow(S)={\$} in LR(0) parser.

If in a state the start symbol of grammar is reduced it is written under \$ symbol as accepted.

NOTE: If in any state both reduced and shifted productions are present or two reduced productions are present it is called a conflict situation and the grammar is not LR grammar.

|   | Action                                                          |                                                                    |                                            | Goto                                               |   |
|---|-----------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------|----------------------------------------------------|---|
|   | a                                                               | b                                                                  | \$                                         | S                                                  | A |
| 0 | S <sub>3</sub> (in state 0 goto operation on a goes to state 3) | S <sub>4</sub>                                                     |                                            | 1 (in state 0 goto operation on S goes to state 1) | 2 |
| 1 |                                                                 |                                                                    | Accepted (S is reduced to string accepted) |                                                    |   |
| 2 | S <sub>3</sub>                                                  | S <sub>4</sub>                                                     |                                            |                                                    | 5 |
| 3 | S <sub>3</sub>                                                  | S <sub>4</sub>                                                     |                                            |                                                    | 6 |
| 4 | R <sub>3</sub> ( in state 4 production 3 is reduced)            | R <sub>3</sub> (reduced entry written under all the non terminals) | R <sub>3</sub>                             |                                                    |   |
| 5 | R <sub>1</sub>                                                  | R <sub>1</sub>                                                     | R <sub>1</sub>                             |                                                    |   |
| 6 | R <sub>2</sub>                                                  | R <sub>2</sub>                                                     | R <sub>2</sub>                             |                                                    |   |

NOTE:

1. Two reduced productions in one state – RR conflict.
2. One reduced and one shifted production in one state – SR conflict.

If no SR or RR conflict present in the parsing table then the grammar is LR(0) grammar.

In above grammar no conflict so it is LR(0) grammar.

## 12. SLR Parsing.

SLR means simple LR. A grammar for which an SLR parser can be constructed is said to be an SLR grammar.

- a. SLR is a type of LR parser with small parse tables and a relatively simple parser generator algorithm. It is quite efficient at finding the single correct bottom up parse in a single left to right scan over the input string, without guesswork or backtracking.
- b. The parsing table has two states (action, Go to). The parsing table has four values:
  - i. Shift S, where S is a state
  - ii. reduce by a grammar production
  - iii. accept, and
  - iv. [error Example:](#)

$E \rightarrow E + T \mid T$   
 $T \rightarrow TF \mid F$   
 $F \rightarrow F * \mid a \mid b$   
 Augmented  
 grammar:  $E' \rightarrow$   
 $.E \text{ Closure}(I)$

|                                                                                                                                                                                |                                                                               |                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| $I_0 :$<br>$E' \rightarrow .E$<br>$E \rightarrow .E + T$<br>$E \rightarrow .T$<br>$T \rightarrow .TF$<br>$T \rightarrow .F$<br>$F \rightarrow .F$<br>$* F \rightarrow$<br>$.a$ | $I_1 : \text{Go to } (I_0, E)$<br>$E' \rightarrow E.$<br>$E \rightarrow E.+T$ | $I_2 : \text{Go to } (I_0, T)$<br>$E \rightarrow T.$<br>$T \rightarrow$<br>$T.F$<br>$F \rightarrow .F$<br>$*$<br>$F \rightarrow .a$<br>$F \rightarrow .b$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                     |                                                                              |                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------|
| $F \rightarrow .b$                                                                                                                                                                  |                                                                              |                                                         |
| $I_3 : \text{Go to } (I_0, F)$<br>$T \rightarrow F.$<br>$F \rightarrow F.*$                                                                                                         | $I_4 : \text{Go to } (I_0, a)$<br>$F \rightarrow a.$                         | $I_5 : \text{Go to } (I_0, b)$<br>$F \rightarrow b.$    |
| $I_6 : \text{Go to } (I_1, +)$<br>$E \rightarrow$<br>$E+.T$<br>$T \rightarrow .TF$<br>$T \rightarrow .F$<br>$F \rightarrow .F^*$<br>$F \rightarrow$<br>$.F^*$<br>$F \rightarrow .a$ | $I_7 : \text{Go to } (I_2, F)$<br>$T \rightarrow TF.$<br>$F \rightarrow F.*$ | $I_8 : \text{Go to } (I_3, ^*)$<br>$F \rightarrow F^*.$ |

|                                                                                                                                                     |  |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| $F \rightarrow .b$                                                                                                                                  |  |  |
| $I_9$ : Go to( $I_6, T$ )<br>$E \rightarrow E +$<br>$T. T \rightarrow T.F$<br>$F \rightarrow .F$<br>$*$<br>$F \rightarrow .a$<br>$F \rightarrow .b$ |  |  |

Table 3.1.15. Canonical LR(0) collection

Follow:

Follow ( E ) : {+, \$}

Follow ( T ) : {+, a, b, \$}

Follow ( F ) : {+, \*, a, b, \$} SLR

parsing table :

| Action |       |       |       |       |        | Go to |   |   |
|--------|-------|-------|-------|-------|--------|-------|---|---|
| state  | +     | *     | a     | b     | \$     | E     | T | F |
| 0      |       |       | $S_4$ | $S_5$ |        | 1     | 2 | 3 |
| 1      | $S_6$ |       |       |       | Accept |       |   |   |
| 2      | $R_2$ |       | $S_4$ | $S_5$ | $R_2$  |       |   | 7 |
| 3      | $R_4$ | $S_8$ | $R_4$ | $R_4$ | $R_4$  |       |   |   |
| 4      | $R_6$ | $R_6$ | $R_6$ | $R_6$ | $R_6$  |       |   |   |
| 5      | $R_6$ | $R_6$ | $R_6$ | $R_6$ | $R_6$  |       |   |   |
| 6      |       |       | $S_4$ | $S_5$ |        |       | 9 | 3 |
| 7      | $R_3$ | $S_8$ | $R_3$ | $R_3$ | $R_3$  |       |   |   |
| 8      | $R_5$ | $R_5$ | $R_5$ | $R_5$ | $R_5$  |       |   |   |
| 9      | $R_1$ |       | $S_4$ | $S_5$ | $R_1$  |       |   | 7 |

Table 3.1.16. SLR Parsing table

## 13. CLR Parsing and LALR Parsing Technique.

Example :  $S \rightarrow C C$

$C \rightarrow a C \mid d$

Augmented grammar:  $S' \rightarrow .S, \$$

Closure(I)

|                                                                                                                                   |                                                                |                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| $I_0 :$ $S' \rightarrow .S, \$$<br>$S \rightarrow .CC,$<br>$\$$<br>$C \rightarrow .a C, a \mid d$<br>$C \rightarrow .d, a \mid d$ | $I_1 : \text{Go to}(I_0, S)$<br>$S' \rightarrow S. , \$$       | $I_2 : \text{Go to}(I_0, C)$<br>$S \rightarrow C.C, \$$<br>$C \rightarrow .a C,$<br>$\$ C \rightarrow .d, \$$ |
| $I_3 : \text{Go to} (I_0, a)$<br>$C \rightarrow a.C, a \mid d$<br>$C \rightarrow .a C, a \mid$<br>$d$                             | $I_4 : \text{Go to} (I_0, d)$<br>$C \rightarrow d. , a \mid d$ | $I_5 : \text{Go to} (I_2, C)$<br>$S \rightarrow C C., \$$                                                     |

|                                                                                                                  |                                                      |                                                              |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------------------------|
| $C \rightarrow .d, a \mid d$                                                                                     |                                                      |                                                              |
| $I_6$ : Go to ( $I_2, a$ )<br>$C \rightarrow a.C, \$$<br>$C \rightarrow .a C,$<br>$\$ C \rightarrow .d,$<br>$\$$ | $I_7$ : Go to ( $I_2, d$ )<br>$C \rightarrow d., \$$ | $I_8$ : Go to ( $I_3, C$ )<br>$C \rightarrow a C., a \mid d$ |
| $I_9$ : Go to ( $I_6, C$ )<br>$C \rightarrow a C., \$$                                                           |                                                      |                                                              |

Table 3.1.17. Canonical LR(1) collection

Parsing table:

| Action |       |       |        | Go to |   |
|--------|-------|-------|--------|-------|---|
| state  | a     | d     | \$     | S     | C |
| 0      | $S_3$ | $S_4$ |        | 1     | 2 |
| 1      |       |       | Accept |       |   |
| 2      | $S_6$ | $S_7$ |        |       | 5 |
| 3      | $S_3$ | $S_4$ |        |       | 8 |
| 4      | $R_3$ | $R_3$ |        |       |   |
| 5      |       |       | $R_1$  |       |   |
| 6      | $S_6$ | $S_7$ |        |       | 9 |
| 7      |       |       | $R_3$  |       |   |
| 8      | $R_2$ | $R_2$ |        |       |   |
| 9      |       |       | $R_2$  |       |   |

Table 3.1.18. CLR Parsing table

## LALR Parsing.

- LALR is often used in practice because the tables obtained by it are considerably smaller than canonical LR.

Example :  $S \rightarrow CC$

$C \rightarrow aC \mid d$

Augmented grammar:  $S' \rightarrow .S, \$$

Closure(I)

|                                                                                                      |                                                      |                                                                                                            |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| $I_0$ :<br>$S' \rightarrow .S, \$$<br>$S \rightarrow .CC,$<br>$\$$<br>$C \rightarrow .a C, a \mid d$ | $I_1$ : Go to( $I_0, S$ )<br>$S' \rightarrow S., \$$ | $I_2$ : Go to( $I_0, C$ )<br>$S \rightarrow C.C, \$$<br>$C \rightarrow .a C,$<br>$\$ C \rightarrow .d, \$$ |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------|

|                                                                                                                                   |                                                                 |                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|
| $C \rightarrow .d, a \mid d$                                                                                                      |                                                                 |                                                                   |
| $I_3 : \text{Go to } (I_0, a)$<br>$C \rightarrow a.C, a \mid d$<br>$C \rightarrow .a C, a \mid d$<br>$C \rightarrow .d, a \mid d$ | $I_4 : \text{Go to } (I_0, d)$<br>$C \rightarrow d. , a \mid d$ | $I_5 : \text{Go to } (I_2, C)$<br>$S \rightarrow C C., \$$        |
| $I_6 : \text{Go to } (I_2, a)$<br>$C \rightarrow a.C, \$$<br>$C \rightarrow .a C, \$$<br>$C \rightarrow .d, \$$                   | $I_7 : \text{Go to } (I_2, d)$<br>$C \rightarrow d. , \$$       | $I_8 : \text{Go to } (I_3, C)$<br>$C \rightarrow a C. , a \mid d$ |

|                                                           |  |  |
|-----------------------------------------------------------|--|--|
| $I_9 : \text{Go to } (I_6, C)$<br>$C \rightarrow a C, \$$ |  |  |
|-----------------------------------------------------------|--|--|

**Table 3.1.19. Canonical LR(1) collection**

Now we will merge state 3, 6 then 4, 7 and 8, 9.

$I_{36} : C \rightarrow a.C, a \mid d \mid \$$

$C \rightarrow .a C, a \mid d \mid$

$\$ C \rightarrow .d, a \mid d \mid$

$\$$

$I_{47} : C \rightarrow d., a \mid d \mid \$$

$I_{89} : C \rightarrow aC., a \mid d \mid \$$

Parsing table:

| Action |                 |                 |                | Go to |     |
|--------|-----------------|-----------------|----------------|-------|-----|
| State  | a               | d               | \$             | S     | C   |
| 0      | S <sub>36</sub> | S <sub>47</sub> |                | 1     | 2   |
| 1      |                 |                 | Accept         |       |     |
| 2      | S <sub>36</sub> | S <sub>47</sub> |                |       | 5   |
| 36     | S <sub>36</sub> | S <sub>47</sub> |                |       | 8 9 |
| 47     | R <sub>3</sub>  | R <sub>3</sub>  | R <sub>3</sub> |       |     |
| 5      |                 |                 | R <sub>1</sub> |       |     |
| 89     | R <sub>2</sub>  | R <sub>2</sub>  | R <sub>2</sub> |       |     |

**Table 3.1.20. LALR parsing table**

## 14. Syntax Directed Definitions.

- Syntax directed definition is a generalization of context free grammar in which each grammar symbol has an associated set of attributes.
- Types of attributes are,
  - Synthesized attribute
  - Inherited attribute
- Difference between synthesized and inherited attribute,

| No | Synthesized Attribute                                                                                                                 | Inherited attribute                                                                                                                |
|----|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Value of synthesized attribute at a node can be computed from the value of attributes at the children of that node in the parse tree. | Values of the inherited attribute at a node can be computed from the value of attribute at the parent and/or siblings of the node. |

|   |                                                            |                                                                                                         |
|---|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 2 | Pass the information from bottom to top in the parse tree. | Pass the information top to bottom in the parse tree <i>or</i> from left siblings to the right siblings |
|---|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|

Table 3.2.1 Difference between Synthesized and Inherited attribute

## 15. Explain synthesized attributes with example.

**OR Write a syntax directed definition for desk calculator.**

- Value of synthesized attribute at a node can be computed from the value of attributes at the children of that node in the parse tree.
- Syntax directed definition be S- attribute definition.
- A parse tree for an S- evaluating the semantic from the leaves to root. m up,
- An annotated parse tree is a parse tree showing the value of the attribute each node. The process of computing the attribute values at the node is called annotating or decorating the parse tree.
- The syntax directed definition for simple desk calculator is given in table

| Production             | Semantic Rules           |
|------------------------|--------------------------|
| $L \rightarrow En$     | $Print(E.val)$           |
| $E \rightarrow E1 + T$ | $E.val = E1.val + T.val$ |
| $E \rightarrow T$      | $E.val = T.val$          |
| $T \rightarrow T1 * F$ | $T.val = T1.val * F.val$ |
| $T \rightarrow F$      | $T.val = F.val$          |
| $F \rightarrow (E)$    | $F.val = E.val$          |
| $F \rightarrow digit$  | $F.val = digit.lexval$   |

Table 3.2.2 Syntax directed definition of a simple desk calculator

## 16. Explain Inherited Attribute.

- An inherited value at a node in a parse tree is defined in terms of attributes at the parent and/or siblings of the node.
- Convenient way for expressing the dependency of a programming language construct on the context in which it appears.
- We can use inherited attributes to keep track of whether an identifier appears on the left or right side of an assignment to decide whether the address or value of the assignment is needed.

- The inherited attribute distributes type information to the various identifiers in a declaration.

**Example:**

| Production                     | Semantic Rules                                         |
|--------------------------------|--------------------------------------------------------|
| $D \rightarrow T L$            | $L.in = T.type$                                        |
| $T \rightarrow \text{int}$     | $T.type = \text{integer}$                              |
| $T \rightarrow \text{real}$    | $T.type = \text{real}$                                 |
| $L \rightarrow L_1, \text{id}$ | $L_1.in = L.in, \text{addtype}(\text{id.entry}, L.in)$ |
| $L \rightarrow \text{id}$      | $\text{addtype}(\text{id.entry}, L.in)$                |

**Table 3.2.3 Syntax directed definition with inherited attribute**

- Symbol T is associated with a synthesized attribute *type*.
- Symbol L is associated with an inherited attribute *in*.

## 17. Construct a Syntax-Directed Definition that translates arithmetic expressions from infix to prefix notation.

- The grammar that contains all the syntactic rules along with the semantic rules having synthesized attribute only.
- Such a grammar for converting infix operators to prefix is given by using the 'val' as S- attribute.

| Production                   | Semantic Rules                |
|------------------------------|-------------------------------|
| $L \rightarrow E$            | $\text{Print}(E.val)$         |
| $E \rightarrow E+T$          | $E.val = '+' E.val T.val$     |
| $E \rightarrow E-T$          | $E.val = '-' E.val T.val$     |
| $E \rightarrow T$            | $E.val = T.val$               |
| $T \rightarrow T * F$        | $T.val = '*' T.val F.val$     |
| $T \rightarrow T / F$        | $T.val = '/' T.val F.val$     |
| $T \rightarrow F$            | $T.val = F.val$               |
| $F \rightarrow F^P$          | $F.val = '^' F.val P.val$     |
| $F \rightarrow P$            | $F.val = P.val$               |
| $P \rightarrow (E)$          | $P.val = E.val$               |
| $P \rightarrow \text{digit}$ | $P.val = \text{digit.lexval}$ |

**Table 3.2.4 Syntax directed definition for infix to prefix notation**

## Syntax directed translation

In syntax directed translation, along with the grammar we associate some informal notations and these notations are called as semantic rules. So we can say that

Grammar + semantic rule = SDT (syntax directed translation)

In syntax directed translation, every non-terminal can get one or more than one attribute or sometimes 0 attribute depending on the type of the attribute. The value of these attributes is evaluated by the semantic rules associated with the production rule.

- In the semantic rule, attribute is VAL and an attribute may hold anything like a string, a number, a memory location and a complex record
- In Syntax directed translation, whenever a construct encounters in the programming language then it is translated according to the semantic rules define in that particular programming language.

### Example

| Production            | Semantic Rules           |
|-----------------------|--------------------------|
| $E \rightarrow E + T$ | $E.val := E.val + T.val$ |
| $E \rightarrow T$     | $E.val := T.val$         |
| $T \rightarrow T * F$ | $T.val := T.val + F.val$ |
| $T \rightarrow F$     | $T.val := F.val$         |
| $F \rightarrow (F)$   | $F.val := F.val$         |
| $F \rightarrow num$   | $F.val := num.lexval$    |

**E.val** is one of the attributes of E. **num.lexval** is the attribute returned by the lexical analyzer.

## Syntax directed translation scheme

- The Syntax directed translation scheme is a context -free grammar.
- The syntax directed translation scheme is used to evaluate the order of semantic rules.
- In translation scheme, the semantic rules are embedded within the right side of the productions.
- The position at which an action is to be executed is shown by enclosed between braces. It is written within the right side of the production.

### Example

| Production                      | Semantic Rules                  |
|---------------------------------|---------------------------------|
| $S \rightarrow E \$$            | { printE.VAL }                  |
| $E \rightarrow E + E$           | {E.VAL := E.VAL + E.VAL }       |
| $E \rightarrow E * E$           | {E.VAL := E.VAL * E.VAL }       |
| $E \rightarrow (E)$             | {E.VAL := E.VAL }               |
| $E \rightarrow I$               | {E.VAL := I.VAL }               |
| $I \rightarrow I \text{ digit}$ | {I.VAL := 10 * I.VAL + LEXVAL } |

$I \rightarrow \text{digit} \quad \{ I.VAL := LEXVAL \}$

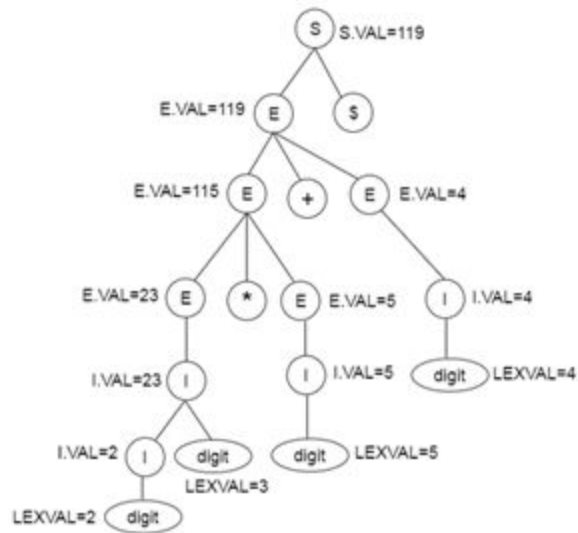
## Implementation of Syntax directed translation

Syntax directed translation is implemented by constructing a parse tree and performing the actions in a left to right depth first order. SDT is implementing by parse the input and produce a parse tree as a result.

### Example

| Production                      | Semantic Rules                       |
|---------------------------------|--------------------------------------|
| $S \rightarrow E \$$            | $\{ \text{print } E.VAL \}$          |
| $E \rightarrow E + E$           | $\{ E.VAL := E.VAL + E.VAL \}$       |
| $E \rightarrow E * E$           | $\{ E.VAL := E.VAL * E.VAL \}$       |
| $E \rightarrow (E)$             | $\{ E.VAL := E.VAL \}$               |
| $E \rightarrow I$               | $\{ E.VAL := I.VAL \}$               |
| $I \rightarrow I \text{ digit}$ | $\{ I.VAL := 10 * I.VAL + LEXVAL \}$ |
| $I \rightarrow \text{digit}$    | $\{ I.VAL := LEXVAL \}$              |

### Parse tree for SDT:



## 18. Dependency graph.

- a. The directed graph that represents the interdependencies between synthesized and inherited attribute at nodes in the parse tree is called dependency graph.
- b. For the rule  $X \rightarrow YZ$  the semantic action is given by  $X.x = f(Y.y, Z.z)$  then synthesized

attribute X.x depends on attributes Y.y and Z.z.

**Algorithm to construct Dependency graph**

```

for each node n in the parse tree do
    for each attribute a of the grammar symbol at n do
        Construct a node in the dependency graph for a;
    for each node n in the parse tree do
        for each semantic rule  $b=f(c_1,c_2,\dots,c_k)$ 
            associated with the production used at n do
            for  $i=1$  to  $k$  do
                construct an edge from the node for  $c_i$  to the node for  $b$ ;

```

**Example:**

$E \rightarrow E1 + E2$

$E \rightarrow E1 * E2$

| Production              | Semantic Rules            |
|-------------------------|---------------------------|
| $E \rightarrow E1 + E2$ | $E.val = E1.val + E2.val$ |
| $E \rightarrow E1 * E2$ | $E.val = E1.val * E2.val$ |

Table 3.2.5 semantic rules

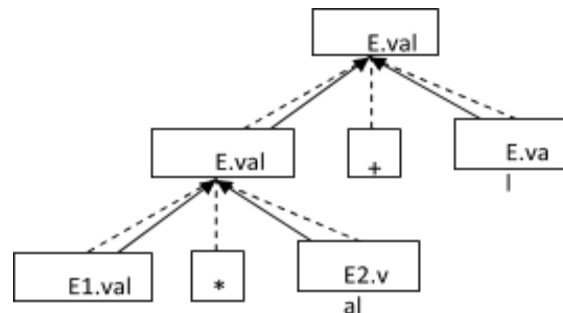


Fig. 3.2.3 Dependency Graph

- c. The synthesized attributes can be represented by .val.
- d. Hence the synthesized attributes are given by E.val, E1.val and E2.val. The dependencies among the nodes are given by solid arrow. The arrow from E1 and E2 show that value of E depends on E1 and E2.

## Unit 4,5,6,7,8

### 1. Basic types of Error.

Error can be classified into mainly two categories,

1. Compile time error
2. Runtime error

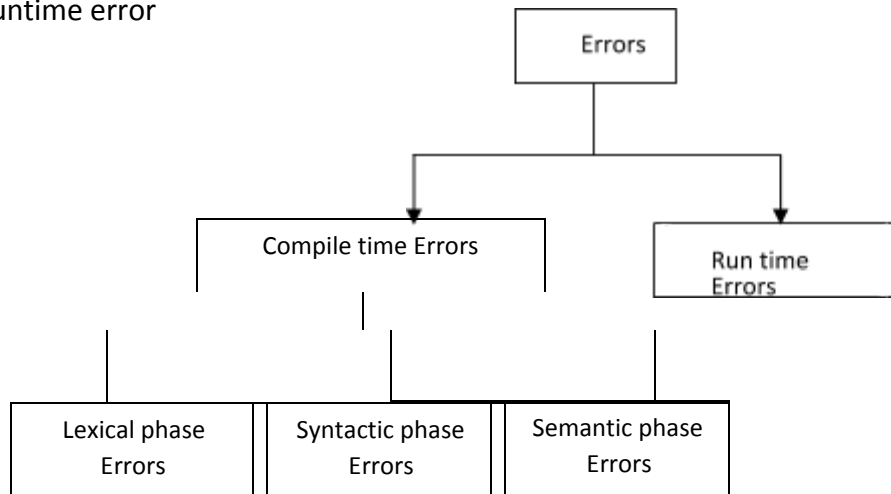


Fig.4.1. Types of Error

#### Lexical Error

This type of errors can be detected during lexical analysis phase. Typical lexical phase errors are,

1. Spelling errors. Hence get incorrect tokens.
2. Exceeding length of identifier or numeric constants.
3. Appearance of illegal characters. Example:

```
fi ( )  
{  
}
```

- In above code 'fi' cannot be recognized as a misspelling of keyword if rather lexical analyzer will understand that it is an identifier and will return it as valid identifier. Thus misspelling causes errors in token formation.

#### Syntax error

These types of error appear during syntax analysis phase of compiler.

Typical errors are:

1. Errors in structure.
  2. Missing operators.
  3. Unbalanced parenthesis.
- The parser demands for tokens from lexical analyzer and if the tokens do not satisfy the grammatical rules of programming language then the syntactical errors get raised.

#### Semantic error

This type of error detected during semantic analysis phase.

Typical errors are:

1. Incompatible types of operands.

2. Undeclared variable.
3. Not matching of actual argument with formal argument.

## 2. Error recovery strategies. OR Ad-hoc and systematic methods.

### 1. Panic mode

- This strategy is used by most parsing methods. This is simple to implement.
- In this method on discovering error, the parser discards input symbol one at a time. This process is continued until one of a designated set of synchronizing tokens is found. Synchronizing tokens are delimiters such as semicolon or end. These tokens indicate an end of input statement.
- Thus in panic mode recovery a considerable amount of input is skipped without checking it for additional errors.
- This method guarantees not to go in infinite loop.
- If there is less number of errors in the same statement then this strategy is best choice.

### 2. Phrase level recovery

- In this method, on discovering an error parser performs local correction on remaining input.
- It can replace a prefix of remaining input by some string. This actually helps parser to continue its job.
- The local correction can be replacing comma by semicolon, deletion of semicolons or inserting missing semicolon. This type of local correction is decided by compiler designer.
- While doing the replacement a care should be taken for not going in an infinite loop.
- This method is used in many error-repairing compilers.

### 3. Error production

- If we have good knowledge of common errors that might be encountered, then we can augment the grammar for the corresponding language with error productions that generate the erroneous constructs.
- If error production is used during parsing, we can generate appropriate error message to indicate the erroneous construct that has been recognized in the input.
- This method is extremely difficult to maintain, because if we change grammar then it becomes necessary to change the corresponding productions.

### 4. Global correction

- We often want such a compiler that makes very few changes in processing an incorrect input string.
- Given an incorrect input string  $x$  and grammar  $G$ , the algorithm will find a parse tree for a related string  $y$ , such that number of insertions, deletions and changes of token

- require to transform x into y is as small as possible.
- Such methods increase time and space requirements at parsing time.
- Global production is thus simply a theoretical concept.

## 19. Intermediate code

Intermediate code is used to translate the source code into the machine code. Intermediate code lies between the high-level language and the machine language.

Fig: Position of intermediate code generator

- If the compiler directly translates source code into the machine code without generating intermediate code then a full native compiler is required for each new machine.
- The intermediate code keeps the analysis portion same for all the compilers that's why it doesn't need a full compiler for every unique machine.
- Intermediate code generator receives input from its predecessor phase and semantic analyzer phase. It takes input in the form of an annotated syntax tree.
- Using the intermediate code, the second phase of the compiler synthesis phase is changed according to the target machine.

## Intermediate representation

Intermediate code can be represented in two ways:

### 1. High Level intermediate code:

High level intermediate code can be represented as source code. To enhance performance of source code, we can easily apply code modification. But to optimize the target machine, it is less preferred.

### 2. Low Level intermediate code

Low level intermediate code is close to the target machine, which makes it suitable for register and memory allocation etc. it is used for machine-dependent optimizations.

## Postfix Notation

- Postfix notation is the useful form of intermediate code if the given language is expressions.
- Postfix notation is also called as 'suffix notation' and 'reverse polish'.
- Postfix notation is a linear representation of a syntax tree.
- In the postfix notation, any expression can be written unambiguously without parentheses.
- The ordinary (infix) way of writing the sum of x and y is with operator in the middle:  $x * y$ . But in the postfix notation, we place the operator at the right end as  $xy *$ .
- In postfix notation, the operator follows the operand.

Example

Production

1.  $E \rightarrow E_1 \text{ op } E_2$
2.  $E \rightarrow (E_1)$
3.  $E \rightarrow \text{id}$

## Semantic Rule

## Program fragment

$E.code = E1.code \parallel E2.code \parallel op$

print op

$E.code = E1.code$

$E.code = id$

print id

## Parse tree and Syntax tree

When you create a parse tree then it contains more details than actually needed. So, it is very difficult to compiler to parse the parse tree. Take the following parse tree as an example:

- In the parse tree, most of the leaf nodes are single child to their parent nodes.
- In the syntax tree, we can eliminate this extra information.
- Syntax tree is a variant of parse tree. In the syntax tree, interior nodes are operators and leaves are operands.
- Syntax tree is usually used when represent a program in a tree structure.

A sentence  $id + id * id$  would have the following syntax tree:

Abstract syntax tree can be represented as:

Abstract syntax trees are important data structures in a compiler. It contains the least unnecessary information.

Abstract syntax trees are more compact than a parse tree and can be easily used by a compiler.

## Three address code

Three-address code is an intermediate code. It is used by the optimizing compilers.

- In three-address code, the given expression is broken down into several separate instructions. These instructions can easily translate into assembly language.
- Each Three address code instruction has at most three operands. It is a combination of assignment and a binary operator.

### Example

**Given Expression:**

**1.  $a := (-c * b) + (-c * d)$**

**Three-address code is as follows:**

$t_1 := -c$   $t_2 := b * t_1$   $t_3 := -c$   $t_4 := d * t_3$   $t_5 := t_2 + t_4$   $a := t_5$

**t is used as registers in the target program.**

**The three address code can be represented in two forms: quadruples and triples.**

## Quadruples

The quadruples have four fields to implement the three address code. The field of quadruples contains the name of the operator, the first source operand, the second source operand and the result respectively.

**Fig: Quadruples field**

### Example

1.  $a := -b * c + d$

Three-address code is as follows:

$$t_1 := -b \qquad t_2 := c + d \qquad t_3 := t_1 * t_2$$

$$a := t_3$$

These statements are represented by quadruples as follows:

|     | Operator | Source 1 | Source 2 | Destination |
|-----|----------|----------|----------|-------------|
|     | uminus   | b        | -        | $t_1$       |
| (   |          |          |          |             |
| 0   |          |          |          |             |
| )   |          |          |          |             |
| (1) | +        | c        | d        | $t_2$       |
| (2) | *        | $t_1$    | $t_2$    | $t_3$       |
| (3) | :=       | $t_3$    | -        | a           |

# Triples

The triples have three fields to implement the three address code. The field of triples contains the name of the operator, the first source operand and the second source operand.

In triples, the results of respective sub-expressions are denoted by the position of expression. Triple is equivalent to DAG while representing expressions.

Fig: Triples field

## Example:

1.  $a := -b * c + d$

Three address code is as follows:

$t_1 := -b$   $t_2 := c + d$   $t_3 := t_1 * t_2$   $a := t_3$

These statements are represented by triples as follows:

|     | Operator | Source 1 | Source 2 |
|-----|----------|----------|----------|
|     | uminus   | b        | -        |
| (   |          |          |          |
| 0   |          |          |          |
| )   |          |          |          |
| (1) | +        | c        | d        |

|     |    |     |     |
|-----|----|-----|-----|
| (2) | *  | (0) | (1) |
| (3) | := | (2) | -   |

## Translation of Assignment Statements

In the syntax directed translation, assignment statement is mainly deals with expressions. The expression can be of type real, integer, array and records.

Consider the grammar

1.  $S \rightarrow id := E$

2.  $E \rightarrow E1 + E2$

3.  $E \rightarrow E1 * E2$

4.  $E \rightarrow (E1)$

5.  $E \rightarrow id$

The translation scheme of above grammar is given below:

|                        |                         |
|------------------------|-------------------------|
| <b>Production rule</b> | <b>Semantic actions</b> |
|------------------------|-------------------------|

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| $S \rightarrow id := E$ | <pre> {p = look_up(id.name);   If p ≠ nil then     Emit (p = E.place)   Else     Error; }</pre> |
| $E \rightarrow E1 + E2$ | <pre> {E.place = newtemp();   Emit (E.place = E1.place '+' E2.place) }</pre>                    |
| $E \rightarrow E1 * E2$ | <pre> {E.place = newtemp();   Emit (E.place = E1.place '*' E2.place) }</pre>                    |
| $E \rightarrow (E1)$    | <pre> {E.place = E1.place}</pre>                                                                |
| $E \rightarrow id$      | <pre> {p = look_up(id.name);   If p ≠ nil then     Emit (p = E.place)   Else     Error; }</pre> |

- The p returns the entry for id.name in the symbol table.

- The Emit function is used for appending the three address code to the output file. Otherwise it will report an error.
- The newtemp() is a function used to generate new temporary variables.
- E.place holds the value of E.

## **Procedures call**

Procedure is an important and frequently used programming construct for a compiler. It is used to generate good code for procedure calls and returns.

### **Calling sequence:**

The translation for a call includes a sequence of actions taken on entry and exit from each procedure. Following actions take place in a calling sequence:

- When a procedure call occurs then space is allocated for activation record.
- Evaluate the argument of the called procedure.
- Establish the environment pointers to enable the called procedure to access data in enclosing blocks.
- Save the state of the calling procedure so that it can resume execution after the call.
- Also save the return address. It is the address of the location to which the called routine must transfer after it is finished.

- Finally generate a jump to the beginning of the code for the called procedure.

Let us consider a grammar for a simple procedure call statement

1.  $S \rightarrow \text{call id}(\text{Elist})$

2.  $\text{Elist} \rightarrow \text{Elist}, E$

3.  $\text{Elist} \rightarrow E$

A suitable transition scheme for procedure call would be:

| Production Rule                              | Semantic Action                                                     |
|----------------------------------------------|---------------------------------------------------------------------|
| $S \rightarrow \text{call id}(\text{Elist})$ | for each item p on QUEUE do<br>GEN (param p)<br>GEN (call id.PLACE) |
| $\text{Elist} \rightarrow \text{Elist}, E$   | append E.PLACE to the end of QUEUE                                  |
| $\text{Elist} \rightarrow E$                 | initialize QUEUE to contain only<br>E.PLACE                         |

Queue is used to store the list of parameters in the procedure call.

## Declarations

When we encounter declarations, we need to lay out storage for the declared variables.

For every local name in a procedure, we create a ST(Symbol Table) entry containing:

1. The type of the name
2. How much storage the name requires

The production:

1.  $D \rightarrow \text{integer}, \text{id}$
2.  $D \rightarrow \text{real}, \text{id}$
3.  $D \rightarrow D1, \text{id}$

A suitable transition scheme for declarations would be:

| Production rule                           | Semantic action                                                              |
|-------------------------------------------|------------------------------------------------------------------------------|
| $D \rightarrow \text{integer}, \text{id}$ | $\text{ENTER}(\text{id.PLACE}, \text{integer})$<br>$D.ATTR = \text{integer}$ |
| $D \rightarrow \text{real}, \text{id}$    | $\text{ENTER}(\text{id.PLACE}, \text{real})$<br>$D.ATTR = \text{real}$       |
| $D \rightarrow D1, \text{id}$             | $\text{ENTER}(\text{id.PLACE}, D1.ATTR)$<br>$D.ATTR = D1.ATTR$               |

ENTER is used to make the entry into symbol table and ATTR is used to trace the data type.

## Storage Organization

- When the target program executes then it runs in its own logical address space in which the value of each program has a location.
- The logical address space is shared among the compiler, operating system and target machine for management and organization. The operating system is used to map the logical address into physical address which is usually spread throughout the memory.

Subdivision of Run-time Memory:

- Runtime storage comes into blocks, where a byte is used to show the smallest unit of addressable memory. Using the four bytes a machine word can form. Object of multibyte is stored in consecutive bytes and gives the first byte address.
- Run-time storage can be subdivide to hold the different components of an executing program:

- 1.** Generated executable code
- 2.** Static data objects
- 3.** Dynamic data-object- heap
- 4.** Automatic data objects- stack

## Activation Record

- Control stack is a run time stack which is used to keep track of the live procedure activations i.e. it is used to find out the procedures whose execution have not been completed.
- When it is called (activation begins) then the procedure name will push on to the stack and when it returns (activation ends) then it will popped.

- Activation record is used to manage the information needed by a single execution of a procedure.
- An activation record is pushed into the stack when a procedure is called and it is popped when the control returns to the caller function.

The diagram below shows the contents of activation records:

**Return Value:** It is used by calling procedure to return a value to calling procedure.

**Actual Parameter:** It is used by calling procedures to supply parameters to the called procedures.

**Control Link:** It points to activation record of the caller.

**Access Link:** It is used to refer to non-local data held in other activation records.

**Saved Machine Status:** It holds the information about status of machine before the procedure is called.

**Local Data:** It holds the data that is local to the execution of the procedure.

**Temporaries:** It stores the value that arises in the evaluation of an expression.

## **Lexical Error**

During the lexical analysis phase this type of error can be detected.

Lexical error is a sequence of characters that does not match the pattern of any token. Lexical phase error is found during the execution of the program.

Lexical phase error can be:

- Spelling error.

- Exceeding length of identifier or numeric constants.
- Appearance of illegal characters.
- To remove the character that should be present.
- To replace a character with an incorrect character.
- Transposition of two characters.

Example:

```
Void main()
```

```
{
```

```
    int x=10, y=20;
```

```
    char * a;
```

```
    a= &x;
```

```
    x= 1xab;
```

```
}
```

In this code, 1xab is neither a number nor an identifier. So this code will show the lexical error.

### Syntax Error

During the syntax analysis phase, this type of error appears. Syntax error is found during the execution of the program.

Some syntax error can be:

- Error in structure
- Missing operators

- Unbalanced parenthesis

When an invalid calculation enters into a calculator then a syntax error can also occurs.

This can be caused by entering several decimal points in one number or by opening brackets without closing them.

For example 1: Using "=" when "==" is needed.

```
16 if (number=200)

17     count << "number is equal to 20";

18 else

19     count << "number is not equal to 200"
```

The following warning message will be displayed by many compilers:

Syntax Warning: assignment operator used in if expression line 16 of program firstprog.cpp

In this code, if expression used the equal sign which is actually an assignment operator not the relational operator which tests for equality.

Due to the assignment operator, number is set to 200 and the expression number=200 are always true because the expression's value is actually 200. For this example the correct code would be:

```
16 if (number==200)
```

Example 2: Missing semicolon:

```
int a = 5    // semicolon is missing
```

Compiler message:

```
ab.java:20: ';' expected
```

```
int a = 5
```

Example 3: Errors in expressions:

```
x = (3 + 5; // missing closing parenthesis ')'
```

```
y = 3 + * 5; // missing argument between '+' and '*'
```

## Semantic Error

During the semantic analysis phase, this type of error appears. These types of error are detected at compile time.

Most of the compile time errors are scope and declaration error. For example: undeclared or multiple declared identifiers. Type mismatched is another compile time error.

The semantic error can arise using the wrong variable or using wrong operator or doing operation in wrong order.

Some semantic error can be:

- Incompatible types of operands
- Undeclared variable
- Not matching of actual argument with formal argument

Example 1: Use of a non-initialized variable:

```
int i;  
  
void f (int m)  
{  
    m=t;  
}
```

In this code, t is undeclared that's why it shows the semantic error.

Example 2: Type incompatibility:

```
int a = "hello";           // the types String and int are not compatible
```

Example 3: Errors in expressions:

```
String s = "...";
```

```
int a = 5 - s;    // the - operator does not support arguments of type String
```

## 20. Symbol Table

Symbol table is an important data structure used in a compiler.

Symbol table is used to store the information about the occurrence of various entities such as objects, classes, variable name, interface, function name etc. it is used by both the analysis and synthesis phases.

The symbol table used for following purposes:

- It is used to store the name of all entities in a structured form at one place.
- It is used to verify if a variable has been declared.
- It is used to determine the scope of a name.
- It is used to implement type checking by verifying assignments and expressions in the source code are semantically correct.

A symbol table can either be linear or a hash table. Using the following format, it maintains the entry for each name.

<symbol name, type, attribute>

For example, suppose a variable store the information about the following variable declaration:

static int salary

then, it stores an entry in the following format:

<salary, int, static>

The clause attribute contains the entries related to the name.

### Implementation

The symbol table can be implemented in the unordered list if the compiler is used to handle the small amount of data.

A symbol table can be implemented in one of the following techniques:

- Linear (sorted or unsorted) list
- Hash table
- Binary search tree

Symbol table are mostly implemented as hash table.

### Operations

The symbol table provides the following operations:

Insert ()

- Insert () operation is more frequently used in the analysis phase when the tokens are identified and names are stored in the table.
- The insert() operation is used to insert the information in the symbol table like the unique name occurring in the source code.

- In the source code, the attribute for a symbol is the information associated with that symbol. The information contains the state, value, type and scope about the symbol.
- The insert () function takes the symbol and its value in the form of argument.

For example:

```
int x;
```

Should be processed by the compiler as:

```
insert (x, int)
```

```
lookup()
```

In the symbol table, lookup() operation is used to search a name. It is used to determine:

- The existence of symbol in the table.
- The declaration of the symbol before it is used.
- Check whether the name is used in the scope.
- Initialization of the symbol.
- Checking whether the name is declared multiple times.

The basic format of lookup() function is as follows:

```
lookup (symbol)
```

This format is varies according to the programming language.

Data structure for symbol table

- A compiler contains two type of symbol table: global symbol table and scope symbol table.
- Global symbol table can be accessed by all the procedures and scope symbol table.

The scope of a name and symbol table is arranged in the hierarchy structure as shown below:

```
int value=10;
```

```
void sum_num()
```

```
{
```

```
    int num_1;
```

```
    int num_2;
```

```
{
```

```
    int num_3;
```

```
    int num_4;
```

```
}
```

```
    int num_5;
```

```
{
```

```
    int_num 6;
```

```
    int_num 7;
```

```
}
```

```
}
```

```
Void sum_id
```

```
{
```

```

    int id_1;

    int id_2;

{
    int id_3;

    int id_4;
}

    int num_5;
}

```

The above grammar can be represented in a hierarchical data structure of symbol tables:

#### Representing Scope Information

In the source program, every name possesses a region of validity, called the scope of that name.

The rules in a block-structured language are as follows:

If a name declared within block B then it will be valid only within B.

If B1 block is nested within B2 then the name that is valid for block B2 is also valid for B1 unless the name's identifier is re-declared in B1.

- These scope rules need a more complicated organization of symbol table than a list of associations between names and attributes.
- Tables are organized into stack and each table contains the list of names and their associated attributes.
- Whenever a new block is entered then a new table is entered onto the stack. The new table holds the name that is declared as local to this block.

- When the declaration is compiled then the table is searched for a name.
- If the name is not found in the table then the new name is inserted.
- When the name's reference is translated then each table is searched, starting from the each table on the stack.

For example:

```
int x;

void f(int m) {
    float x, y;
{
    int i, j;
    int u, v;
}
}

int g (int n)
{
    bool t;
}
```

Fig: Symbol table organization that complies with static scope information rules

The global symbol table contains one global variable and two procedure names. The name mentioned in the sum\_num table is not available for sum\_id and its child tables.

Data structure hierarchy of symbol table is stored in the semantic analyzer. If you want to search the name in the symbol table then you can search it using the following algorithm:

- First a symbol is searched in the current symbol table.

- If the name is found then search is completed else the name will be searched in the symbol table of parent until,
- The name is found or global symbol is searched.

## 21. Storage Allocation

The different ways to allocate memory are:

Static storage allocation

Stack storage allocation

Heap storage allocation

Static storage allocation

- In static allocation, names are bound to storage locations.
- If memory is created at compile time then the memory will be created in static area and only once.
- Static allocation supports the dynamic data structure that means memory is created only at compile time and deallocated after program completion.
- The drawback with static storage allocation is that the size and position of data objects should be known at compile time.
- Another drawback is restriction of the recursion procedure.

Stack Storage Allocation

- In static storage allocation, storage is organized as a stack.
- An activation record is pushed into the stack when activation begins and it is popped when the activation ends.
- Activation record contains the locals so that they are bound to fresh storage in each activation record. The value of locals is deleted when the activation ends.

- It works on the basis of last-in-first-out (LIFO) and this allocation supports the recursion process.

#### Heap Storage Allocation

- Heap allocation is the most flexible allocation scheme.
- Allocation and deallocation of memory can be done at any time and at any place depending upon the user's requirement.
- Heap allocation is used to allocate memory to the variables dynamically and when the variables are no more used then claim it back.
- Heap storage allocation supports the recursion process.

Example:

```
fact (int n)
{
    if (n<=1)
        return 1;
    else
        return (n * fact(n-1));
}

fact (6)
```

The dynamic allocation is as follows:

